Tic-tac-toe Game

Mục tiêu

- Trò chơi thực hiện: TicTacToe
  - Ý nghĩa và cơ chế của các Rule-based AI áp dụng cho trò chơi.
  - Khởi đầu Deep Learning và giải thích các mô hình.
  - Học tăng cường và khả năng tự cải thiện khả năng học chơi trò chơi.
  - AlphaGO - Mô hình tận dụng các thế mạnh của các phương pháp.
  - AlphaZero - Nâng cấp cơ chế đánh giá.
  - Bảng so sánh các độ đo cho các mô hình (Tỉ lệ thắng/thua/hòa, Tốc độ đưa ra quyết định, Không gian lưu trữ khi tính toán trên cây)

Mục tiêu

- Các mô hình cần thực hiện:
  - Rule-based AI
  - MiniMax (Alpha-Beta pruning)
  - MCTS Rollout
  - Neural Network
  - Reinforcement Learning
  - AlphaGO
  - AlphaZero

Rule-based AI

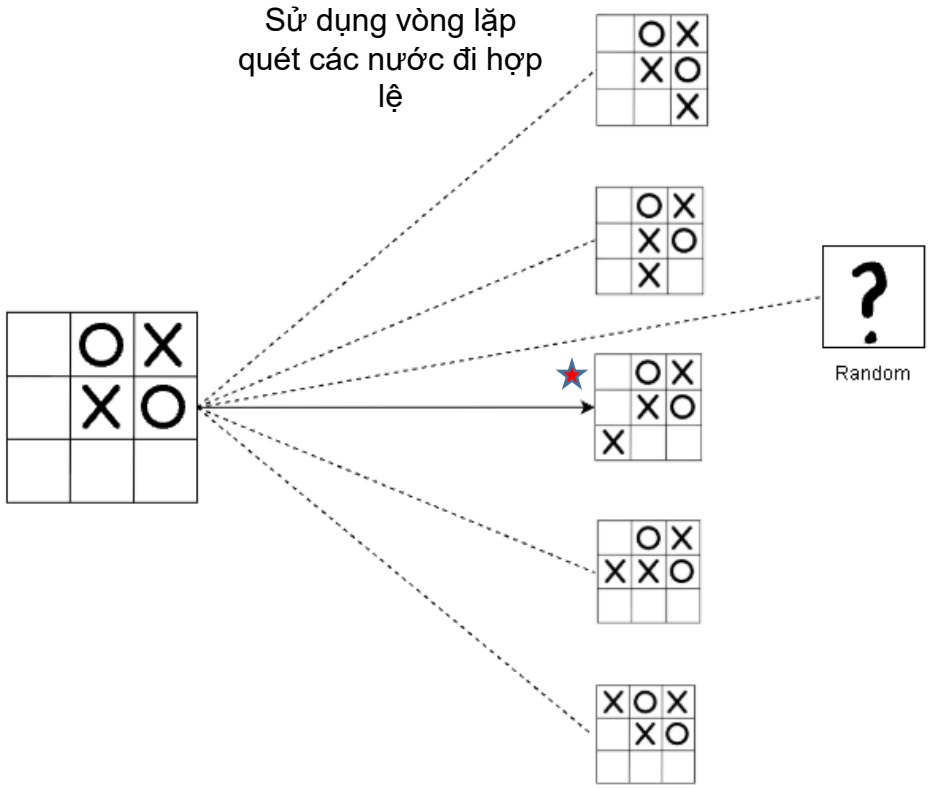
- AI thuần luật chơi (pure rule-based AI)
- Thuật toán Minimax
- Phương pháp tỉa nhánh cho Minimax
- Monte Carlo Tree Search

Rule-based AI

- Thuật toán cố nhìn trước nước đi của đối phương:
  - think1.
  - think2.
  - think3.

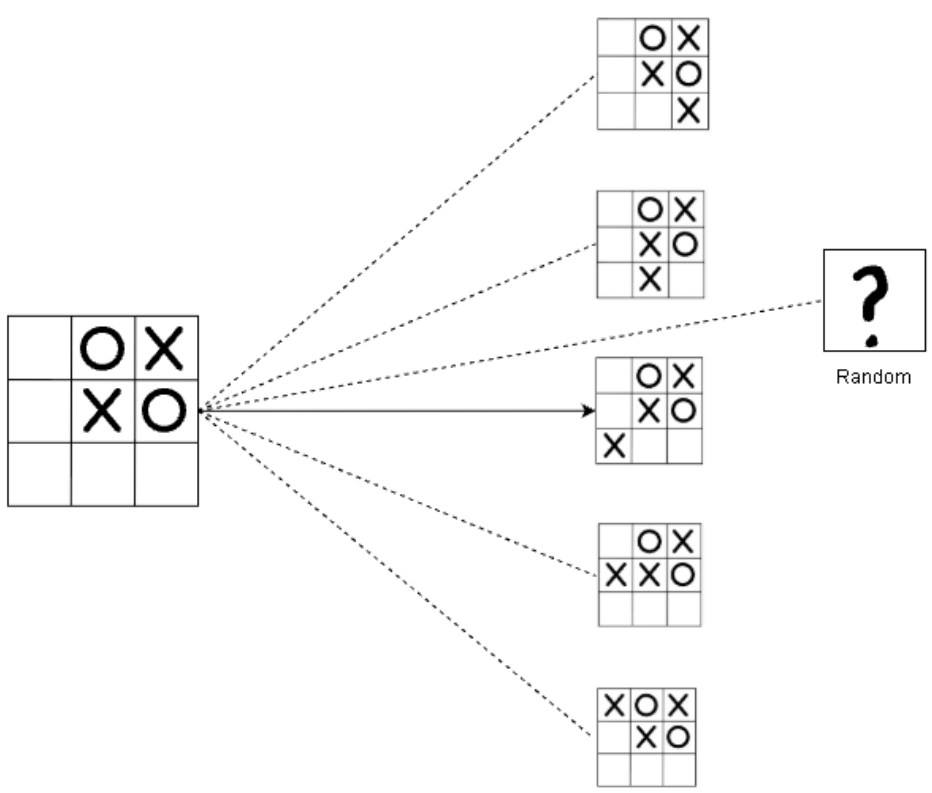
Rule-based AI

- Thuật toán cố nhìn trước nước đi của đối phương:
  - think1.

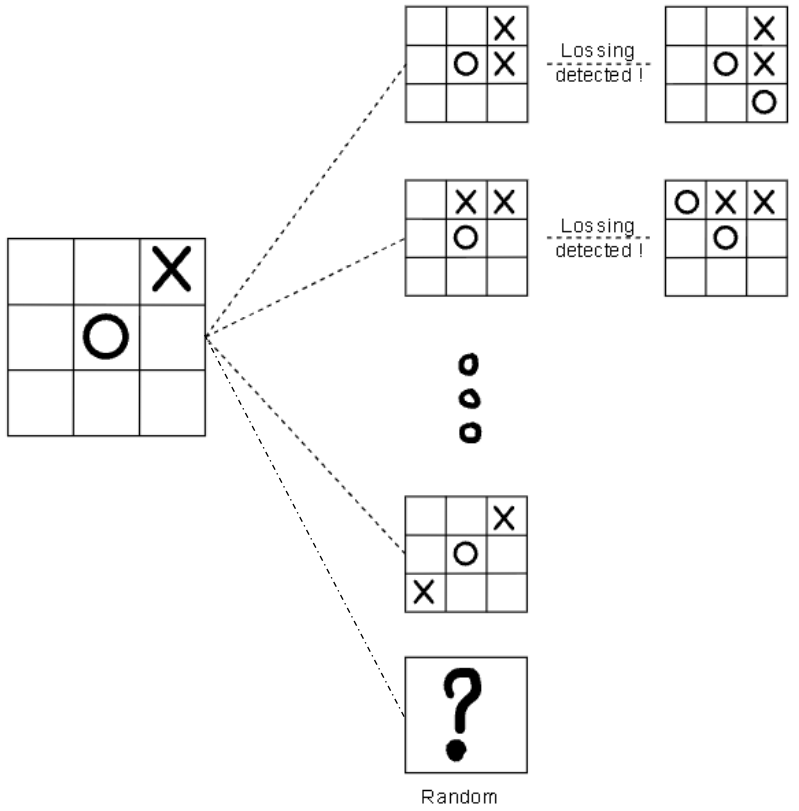


Rule-based AI

- Thuật toán cố nhìn trước nước đi của đối phương:
  - think2.

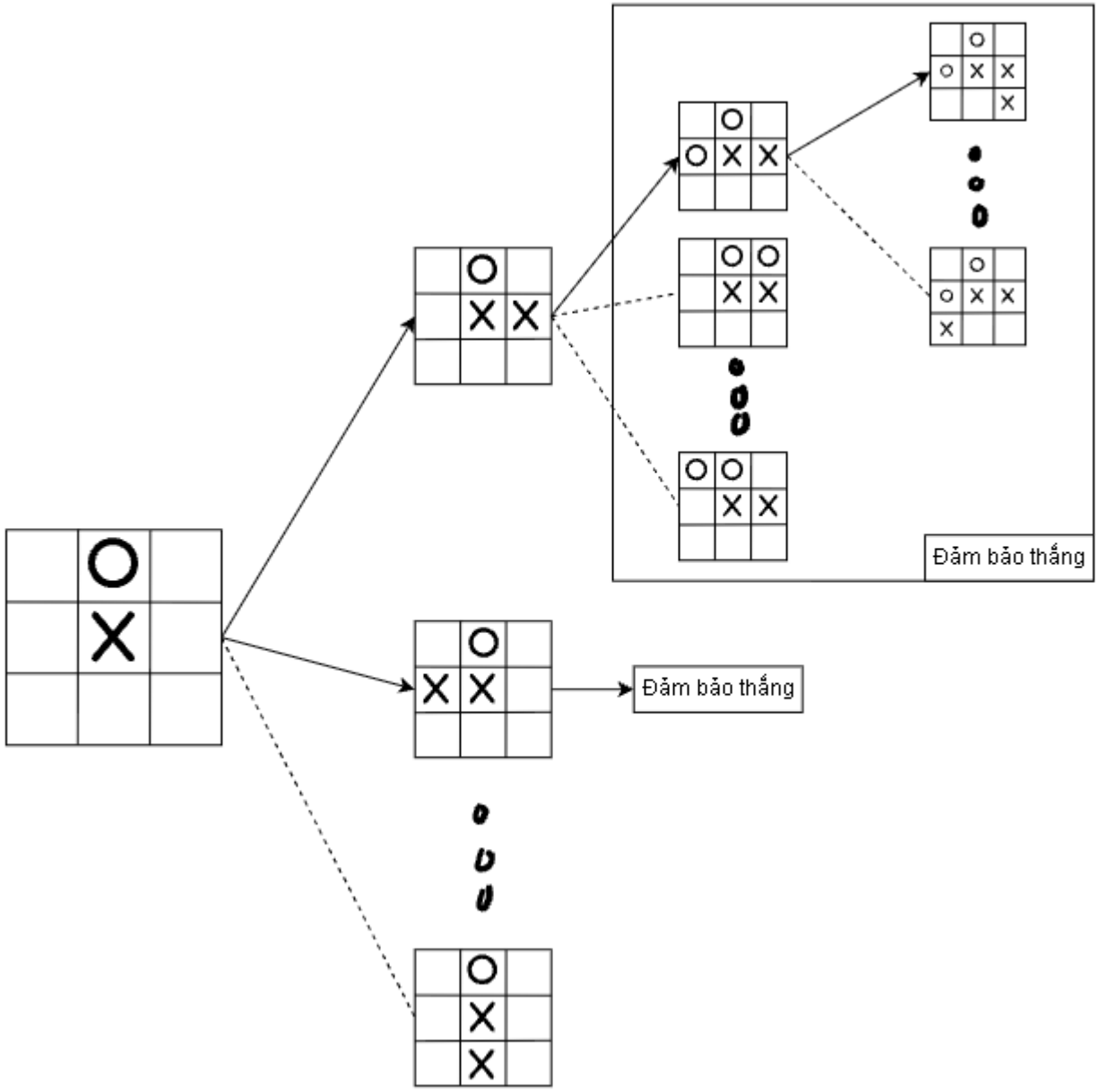


think1



Rule-based AI

- Thuật toán cố nhìn trước nước đi của đối phương:
  - think3: Vẫn giữ 2 vòng lặp nhìn trước 1 và 2 bước của think 3
  - Còn sau khi đã nhìn qua hết các vòng lặp vẫn không tìm được nước đi chiến thắng thì đi ngẫu nhiên



Rule-based AI

- Giới hạn nhìn trước: Độ phức tạp của các vòng lặp khi cố nhìn trước sẽ ngày càng đội lên.
- Một hướng tiếp cận khác: Thử nghiệm một phương pháp chơi thuần theo bộ luật đảm bảo của trò chơi.

Rule-based AI

- Bộ luật được đề xuất bởi Newell và Simon trong cuốn sách "Human Problem Solving".
  - 1. Nếu có 2 quân thẳng hàng (6 thẳng, 2 chéo) → Đi nước thắng.
  - 2. Nếu đối phương có 2 quân thẳng hàng → Chặn lại.
  - 3. Tạo thế đôi: Đánh vào ô có thể tạo thế tấn công đôi.
  - 4. Chặn ô tiềm năng tạo thế đôi của đối phương.
  - 5. Chiếm trung tâm nếu có thể.
  - 6. Nếu đối phương đã chiếm 1 góc → Chiếm ô góc đối diện lại.
  - 7. Chiếm ô cạnh trống.
  - 8. Ô nào còn trống thì đi.

Rule-based AI

- Giới hạn: Chỉ thuần có thể áp dụng được cho trò chơi TicTacToe đổi lại đảm bảo mô hình không thể thua.

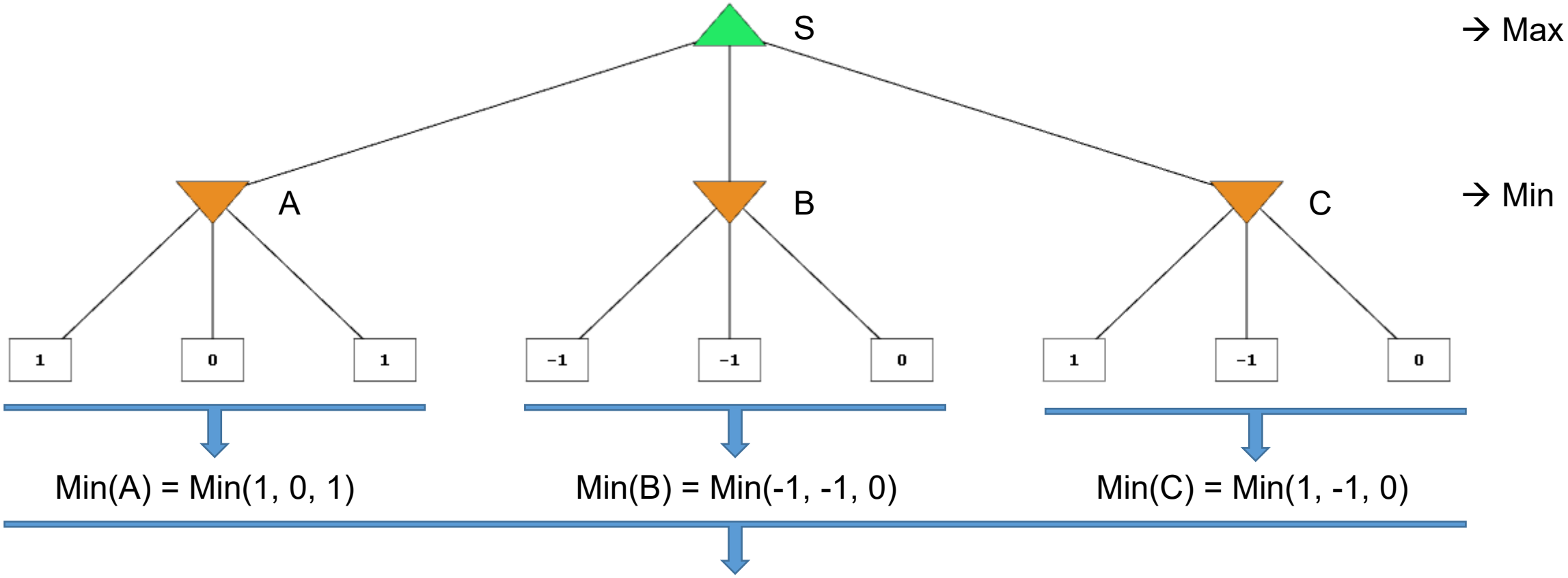
Rule-based AI

- Dẫn đến hướng tiếp cận mới: MiniMax
- Trích dẫn visualizer: https://sarose550.github.io/Minimax-Visualizer/

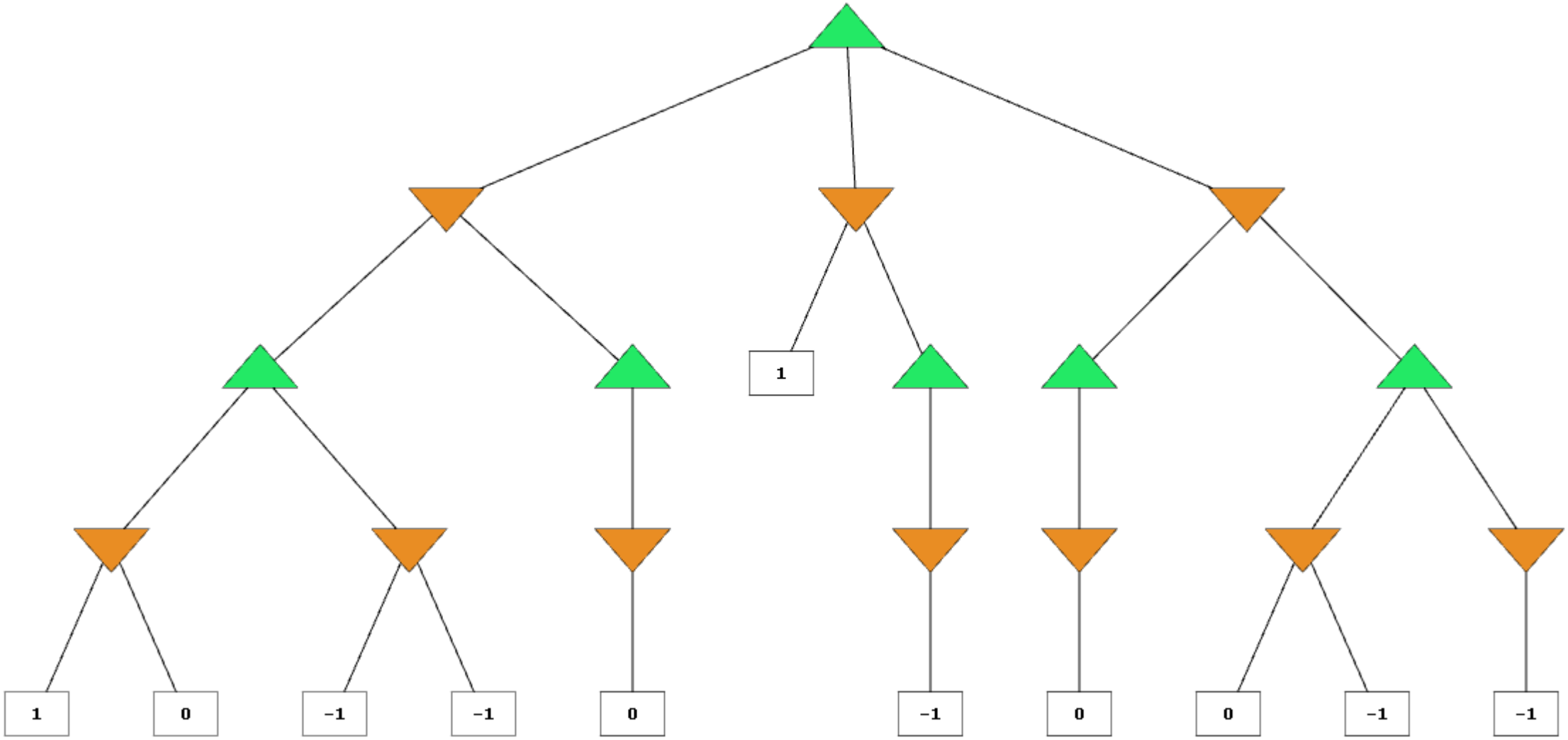
Rule-based AI

- Ý tưởng của MiniMax: Giả định bản thân là người chơi Max và đối phương là người chơi Min. Ta xác định các nhiệm vụ của mỗi bên như sau.
  - Người chơi Max sẽ cố gắng để tối đa khả năng thắng của bản thân trong mọi tình huống.
  - Người chơi Min sẽ cố gắng để người chơi Max bị giảm đi khả năng thắng hoặc sẽ kéo đến thua nếu có cơ hội.

Rule-based AI



Rule-based AI



→ Max

→ Min

→ Max

→ Min

Rule-based AI

- Giới hạn: thời gian để đưa ra một nước đi có thời gian tính toán khá lớn
- Nước đi đầu tiên trong trạng thái đầu tiên mất 35s.

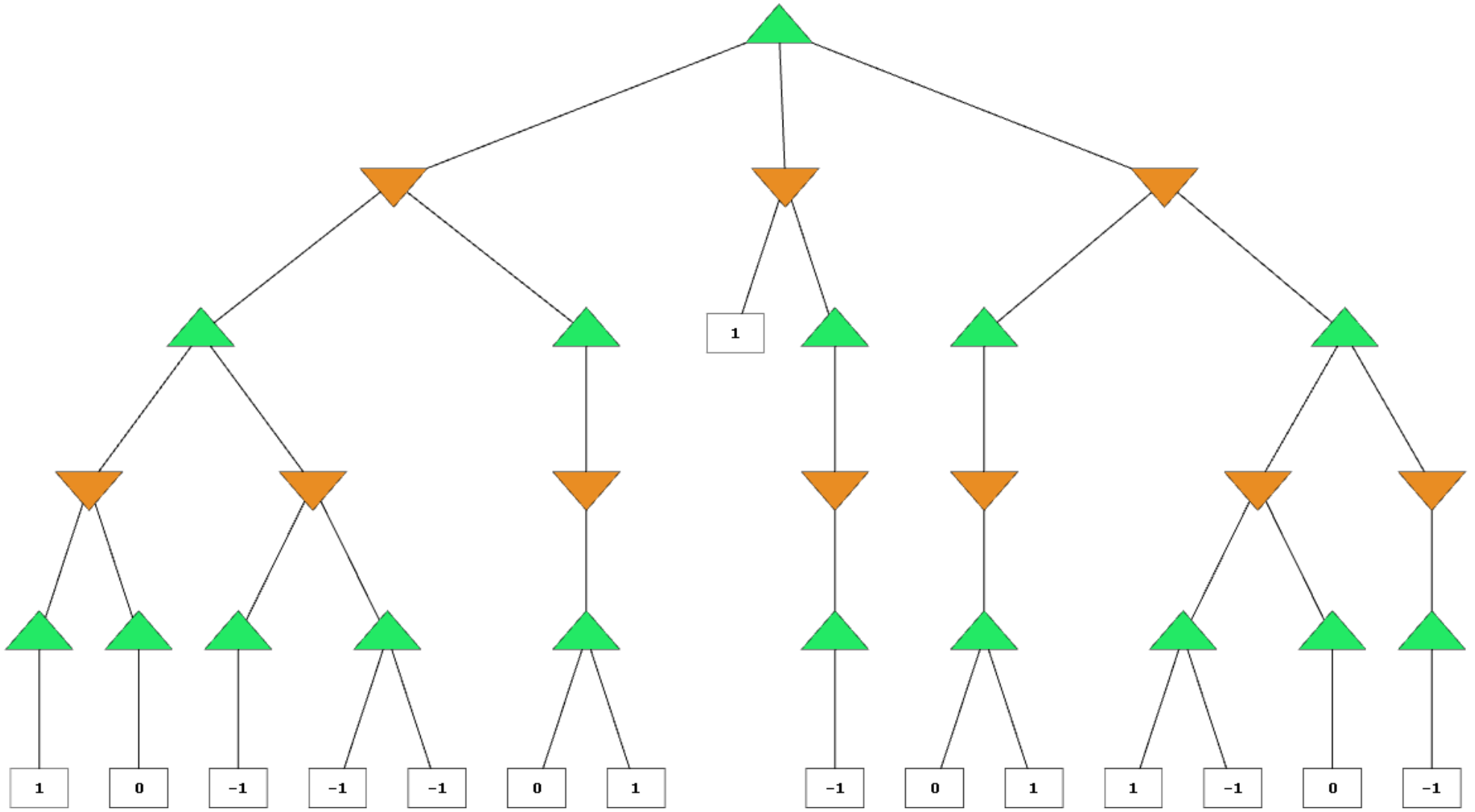
Rule-based AI

- Dẫn đến các phương pháp để ta tỉa nhánh của cây:
  - Depth Pruning.
  - Alpha-Beta Pruning.

Rule-based AI

- Depth Pruning: Đặt ra một giới hạn duyệt cây đến một độ sâu nhất định và ta ước lượng giá trị cuối cùng của bàn cờ nhờ heuristic

Rule-based AI



Rule-based AI

- Giới hạn: Rủi ro không đảm bảo đi hết các trường hợp dẫn đến phần heuristic có khả năng sai sót.

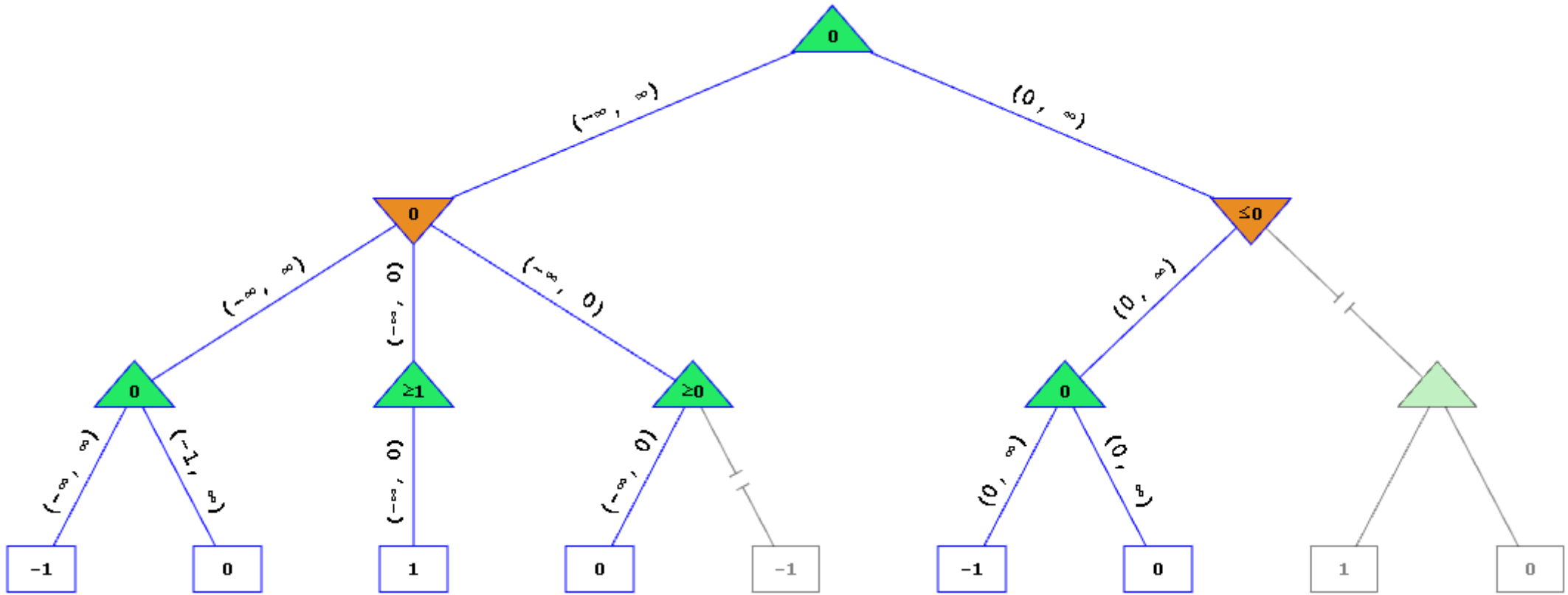
Rule-based AI

- Alpha-Beta Pruning: Vẫn đảm bảo duyệt qua đầy đủ các cây, nhưng giờ đây ta có cách để đảm bảo bỏ bớt đi các nhánh không cần thiết.

Rule-based AI

- Ý tưởng của phương pháp tỉa:

Rule-based AI



Rule-based AI

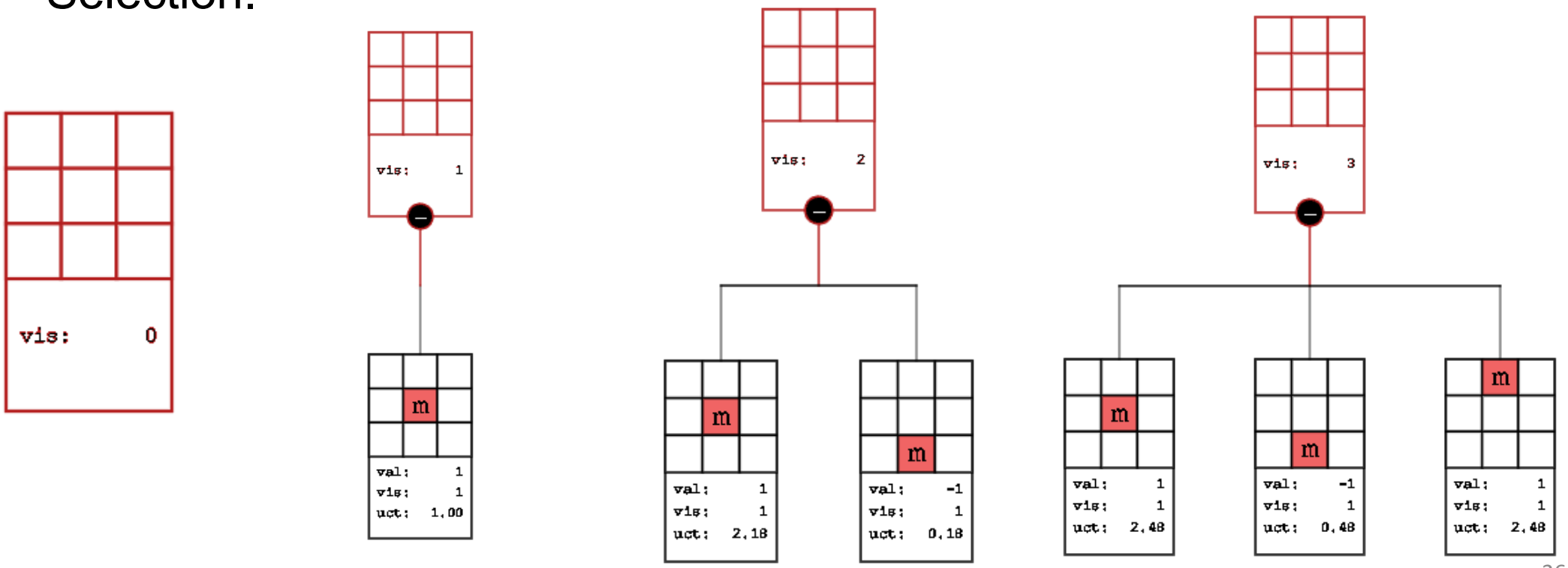
- Hướng tiếp cận khác: Monte Carlo Tree Search
- Trích dẫn visualizer: https://vgarciasc.github.io/mcts-viz/

Rule-based AI

- Quy tắc của MCTS: Là thuật toán có khả năng độc lập tri thức trò chơi (ngoại trừ luật chơi) thông qua cách duyệt cây các trạng thái của trò chơi.
- Gồm 4 bước với số lần lặp lại cho mỗi lần duyệt cây ứng với số lượng rollout ta quy định mỗi lần huấn luyện.
- Hành động tốt nhất được đưa ra cho 1 trạng thái sau khi chạy hết số iteration rollout.

Rule-based AI

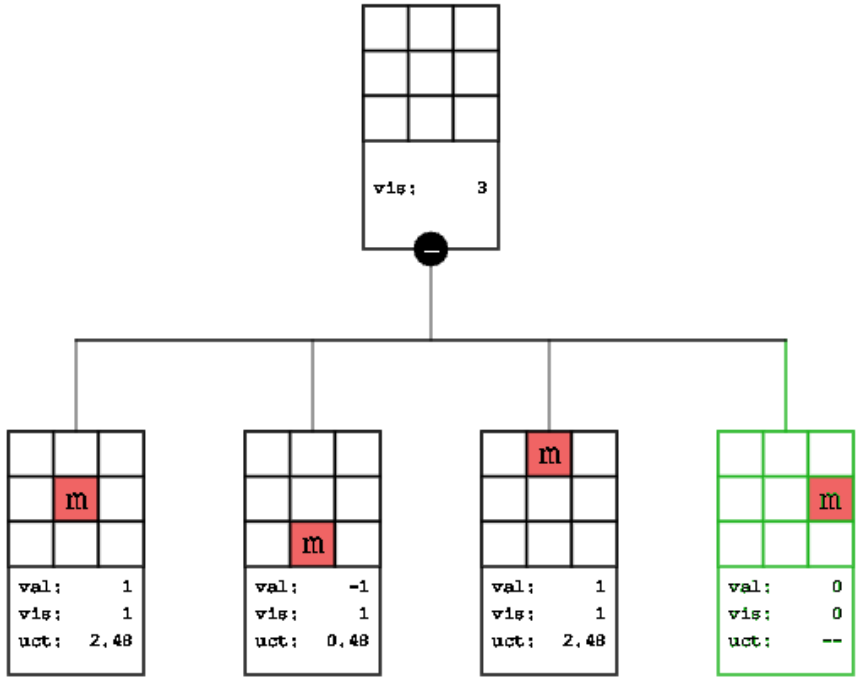
- Trong một vòng lặp rollout để ta duyệt cây tìm kiếm có 4 giai đoạn chính:
  - Selection.



Rule-based AI

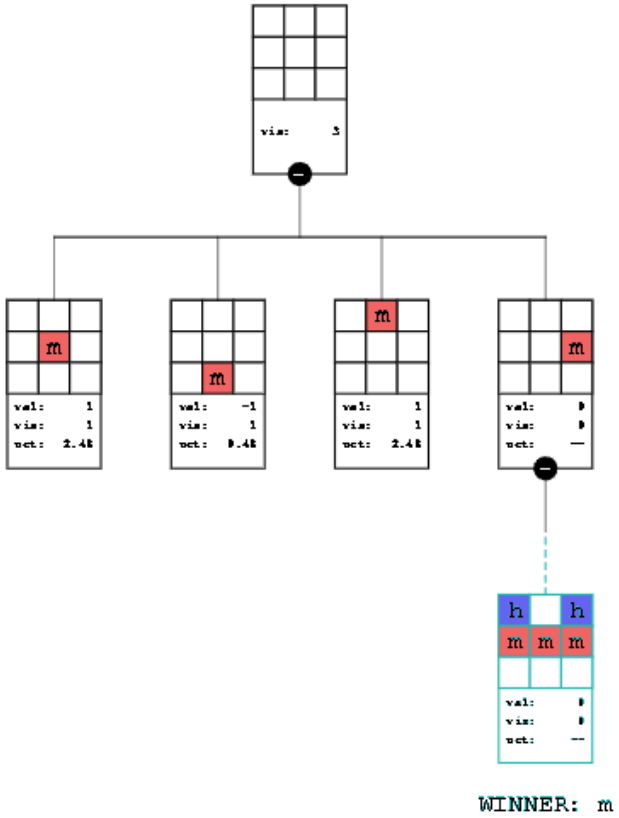
- Expansion.





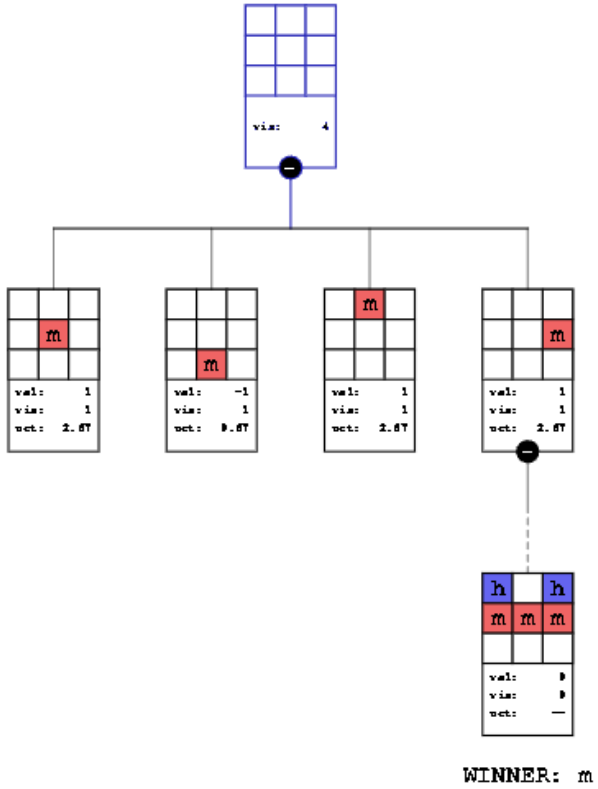
Rule-based AI

- Simulation (Rollout).

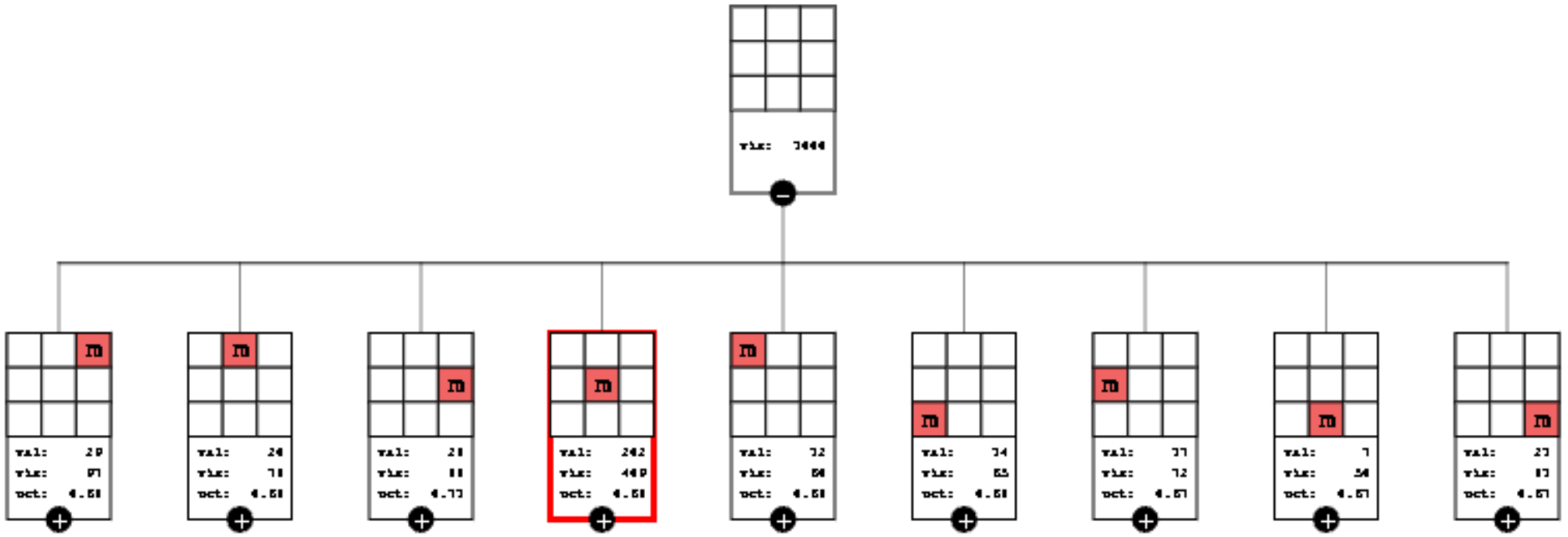


Rule-based AI

- Backpropagate.

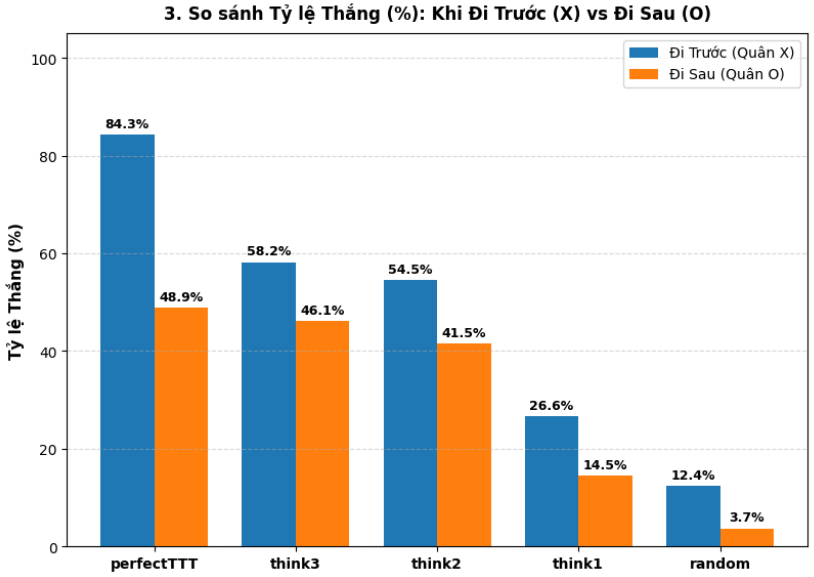


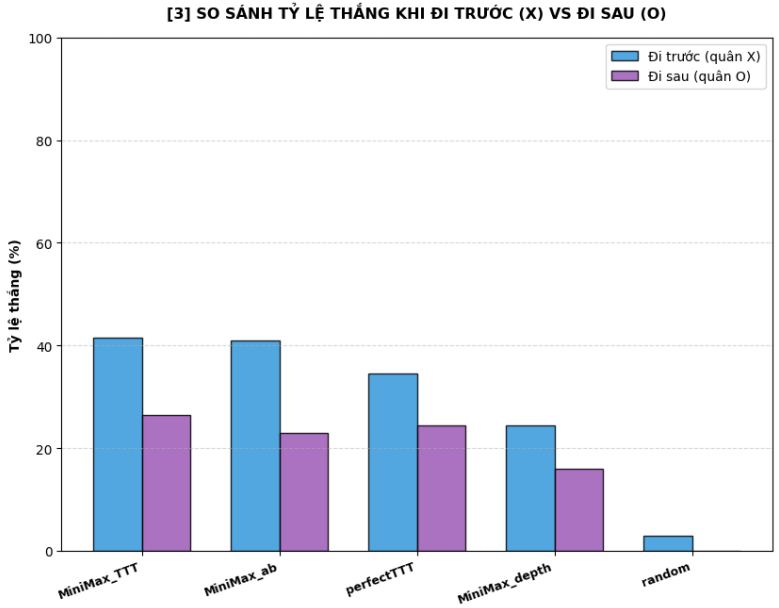
Rule-based AI

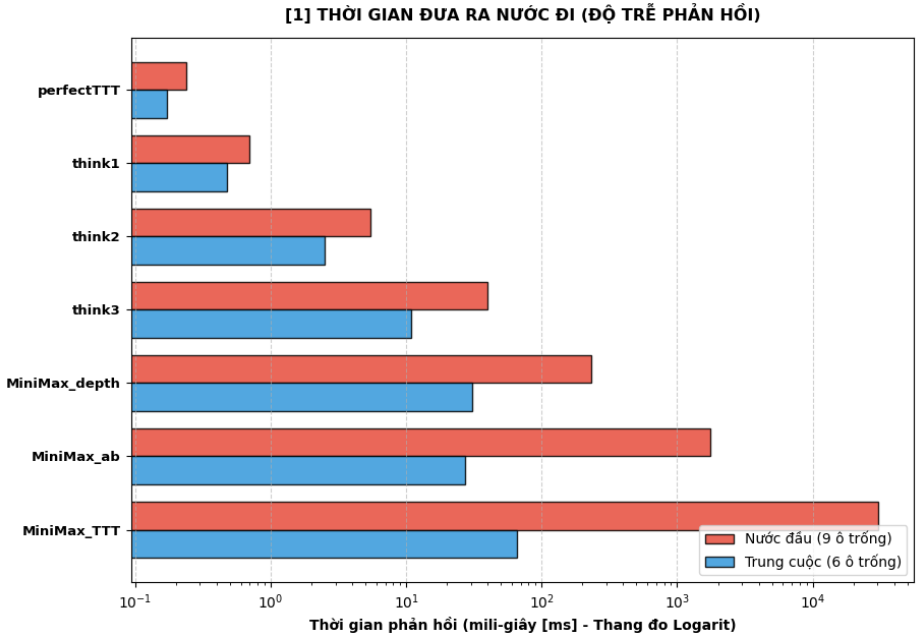


Rule-based AI

- Kiểm nghiệm giữa các thuật toán Rule-based AI:

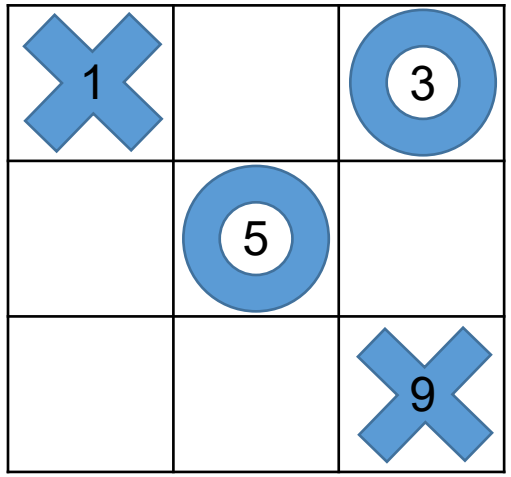






Rule-based AI

- Kiểm nghiệm giữa các thuật toán Rule-based AI:



	Phương pháp	Nước đi đề xuất	Thời gian (ms)
0	MiniMax_TTT	Ô 7	12.480
1	MiniMax_depth (d=3)	Ô 7	8.922
2	MiniMax_ab	Ô 7	8.079

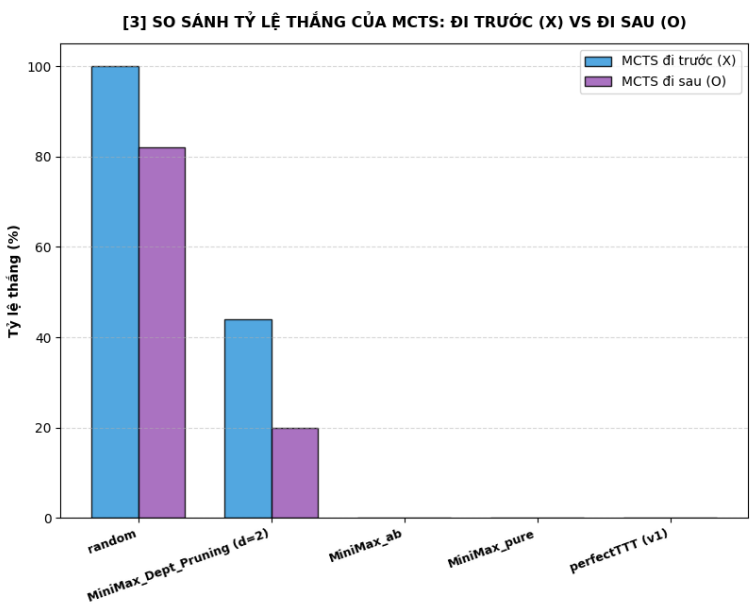
Rule-based AI

- Kiểm nghiệm giữa các thuật toán Rule-based AI:

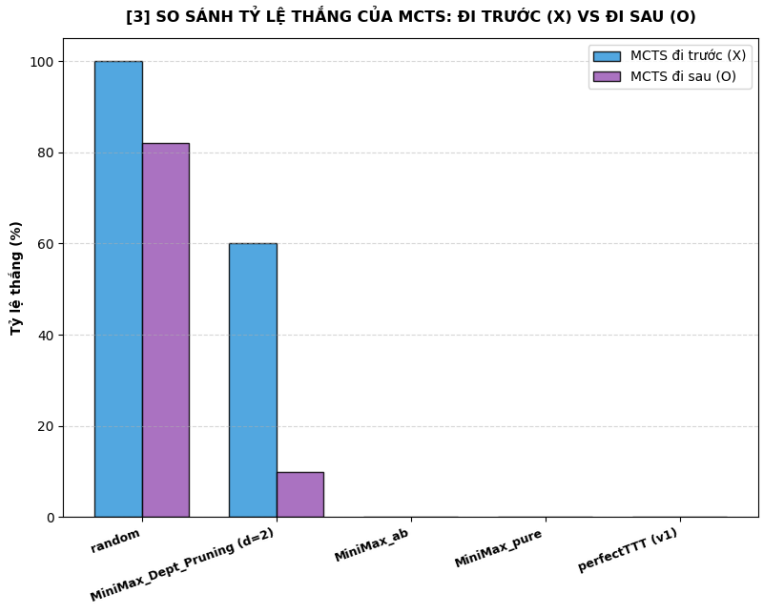
	Mô hình AI	Nước đầu (ms)	Trung cuộc (ms)	Tàn cuộc (ms)	TB (ms)	Nút duyệt (Nước đầu)	Nút duyệt (Trung cuộc)	Độ sâu Call Stack	Không gian O(.)
0	perfectTTT	0.236	0.170	0.063	0.156	1	1	1	O(1)
1	think1	0.698	0.476	0.177	0.450	9	6	1	O(b) = O(9)
2	think2	5.469	2.487	0.206	2.721	81	36	2	O(b^2) = O(81)
3	think3	39.724	10.941	0.200	16.955	585	156	3	O(b^3) = O(729)
4	MiniMax_depth	232.660	30.801	0.467	87.976	73	31	2	O(d) = O(2)
5	MiniMax_ab	1765.043	27.121	0.391	597.518	18296	278	9	O(d) = O(9)
6	MiniMax_TTT	30669.823	65.660	0.398	10245.294	549945	1172	9	O(d) = O(9)

Rule-based AI

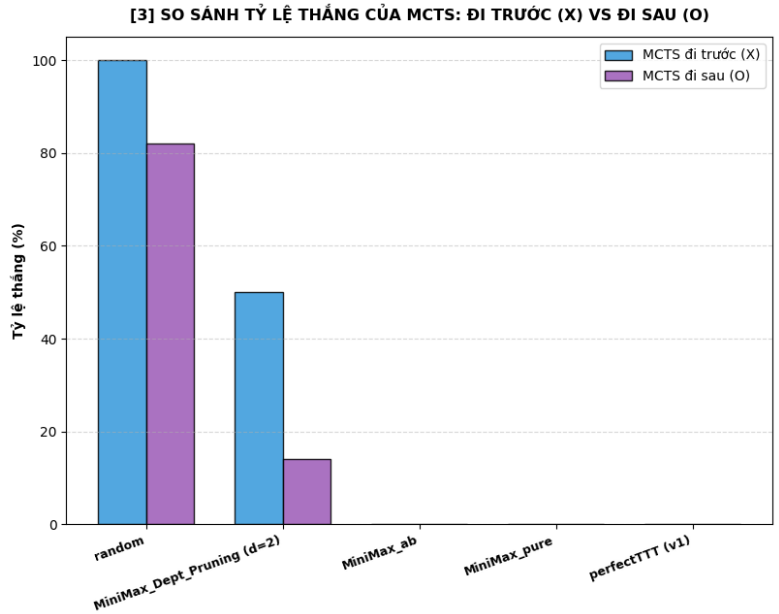
- Kiểm nghiệm giữa các thuật toán Rule-based AI:



n_rollouts = 80



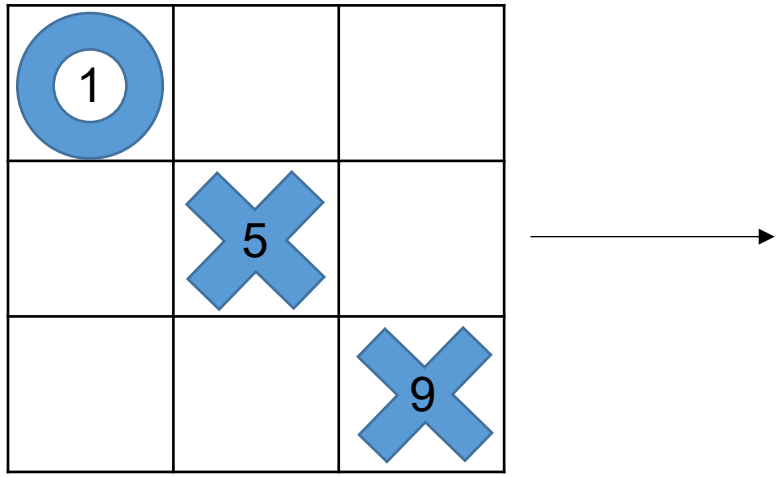
n_rollouts = 100



n_rollouts = 200

Rule-based AI

- Kiểm nghiệm giữa các thuật toán Rule-based AI:



	Mô hình	Số Rollouts	Nước cờ đề xuất	Thời gian (giây)
0	MCTSv1	100	Ô số 7	0.0095
1	MCTSv2	500	Ô số 3	0.0419
2	MCTSv3	1000	Ô số 3	0.0749
3	MCTSv4	1500	Ô số 7	0.1132

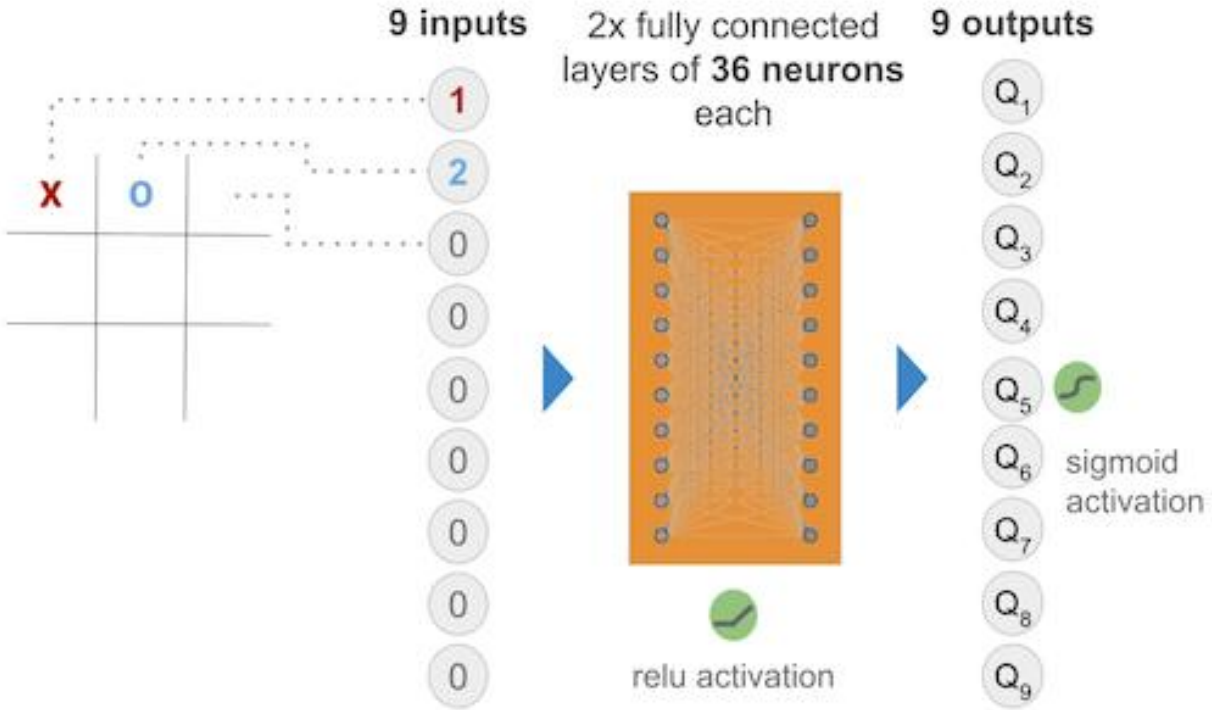
Bảng so sánh tốc độ đưa ra hành động mỗi nước đi của các phiên bản MCTS khác nhau

Deep Learning

- Phương pháp của Deep Learning
- Cách để hệ thống nhận diện trạng thái trò chơi
- Học có giám sát với 2 model phục vụ thiết kế (fast/strong)
- Cải tiến cho MCTS trong tìm kiếm cây game TicTacToe
- Link tham khảo: https://the-mvm.github.io/deep-q-learning-tic-tac-toe.html

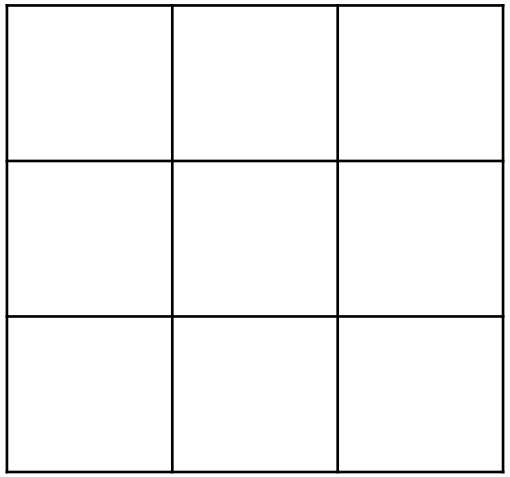
Deep Learning

- Ý tưởng của phương pháp: Sử dụng cách nhận diện trò chơi và luật chơi, kèm theo các lớp tính toán gọi là node để mô hình ghi nhớ cách chơi.



Deep Learning

- Để học chơi phải nhận diện trò chơi:
  - Các giá trị chính: 1(ô do mô hình chiếm), 0 (ô còn trống), -1 (ô do đối phương chiếm) → Khi mô hình chơi với O thì toàn bộ giá trị bàn cờ nhân (-1).
  - Input Shape: Xử lý như hình ảnh 2D (3, 3, 1).



Deep Learning

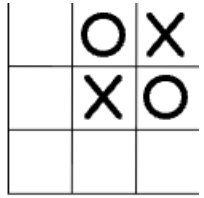
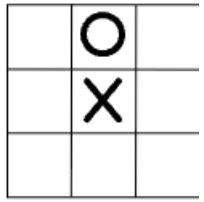
- 2 mô hình huấn luyện có giám sát:
  - Fast_policy_network.
  - Strong_policy_network.
- Mỗi mô hình phục vụ 2 giai đoạn khác nhau khi implement AlphaGO Algorithm.

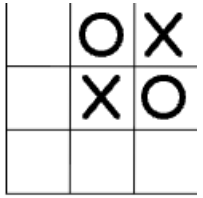
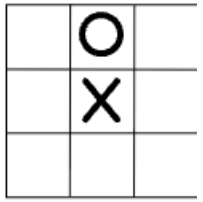
Deep Learning

- Có nhiều thành phần và công đoạn để chú ý đến:
  - Data
  - Preprocess
  - Training & Testing
  - Output & Benchmark

Deep Learning

- Data: Labeled Input & Unlabeled Input
  - Labeled Input: Mẫu có bao gồm nhãn đáp án. X→Y
  - Unlabeled Input: Mẫu không bao gồm nhãn đáp án. X→?

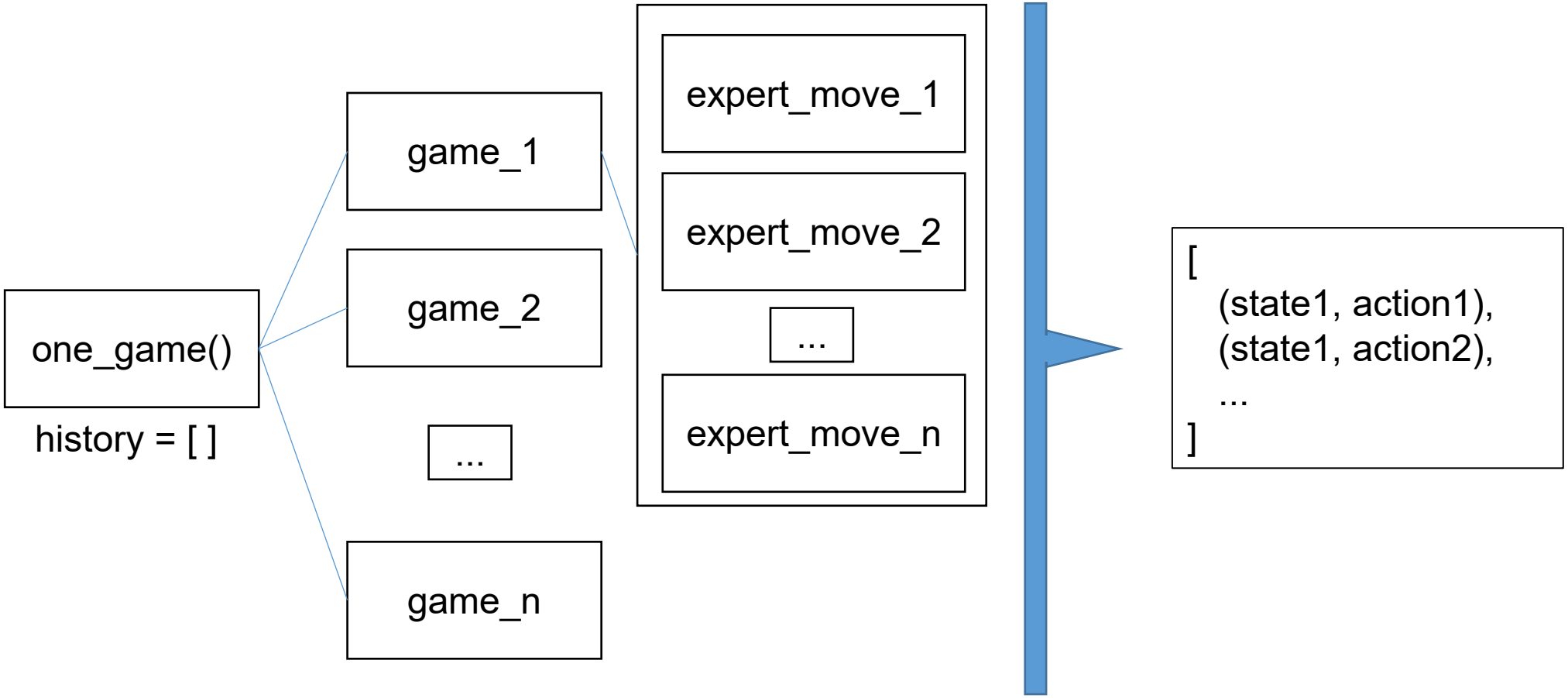
Input X	Label Y
	Ô 7
	Ô 1

Input X



→ Tùy thuộc vào bài toán khác nhau mà dùng loại dữ liệu khác nhau

Deep Learning

- Trong tài liệu dữ liệu được tạo theo cấu trúc.

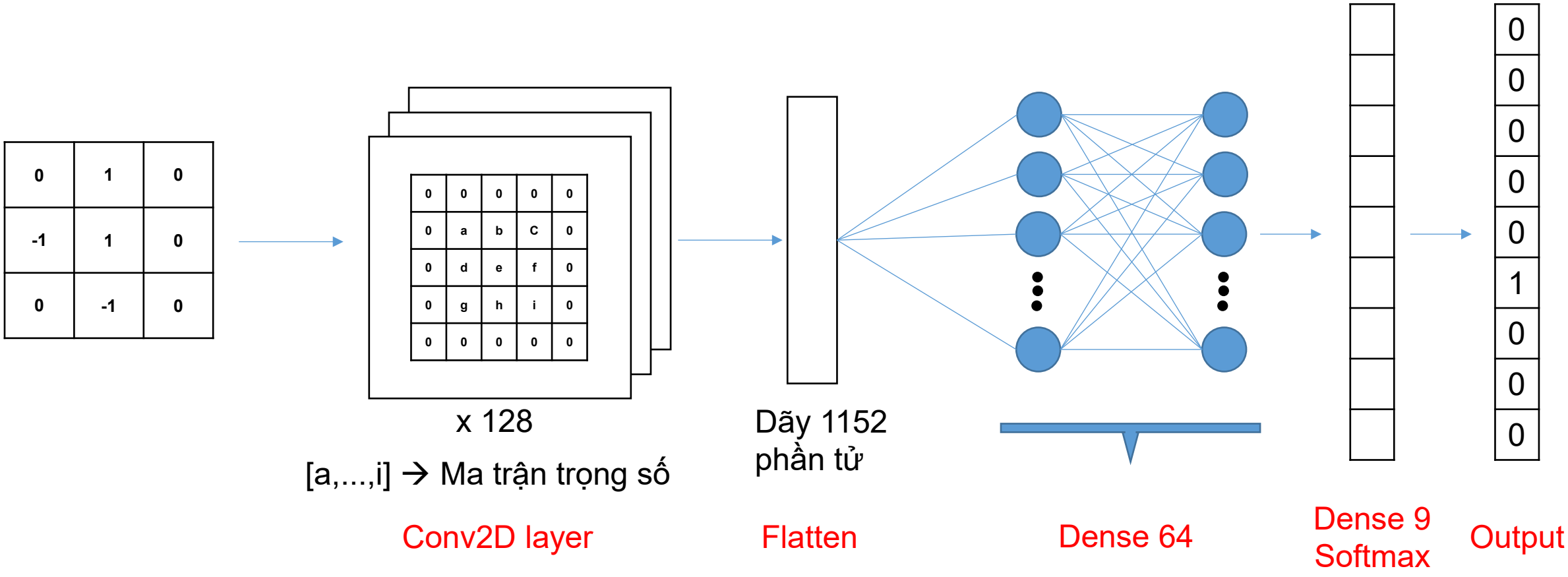


Deep Learning

- 2 mô hình huấn luyện có giám sát:
  - Fast_policy_network.
  - Strong_policy_network.
- Mỗi mô hình phục vụ 2 giai đoạn khác nhau khi implement AlphaGO Algorithm.

Deep Learning

- Kiến trúc của Fast Policy Network



Deep Learning

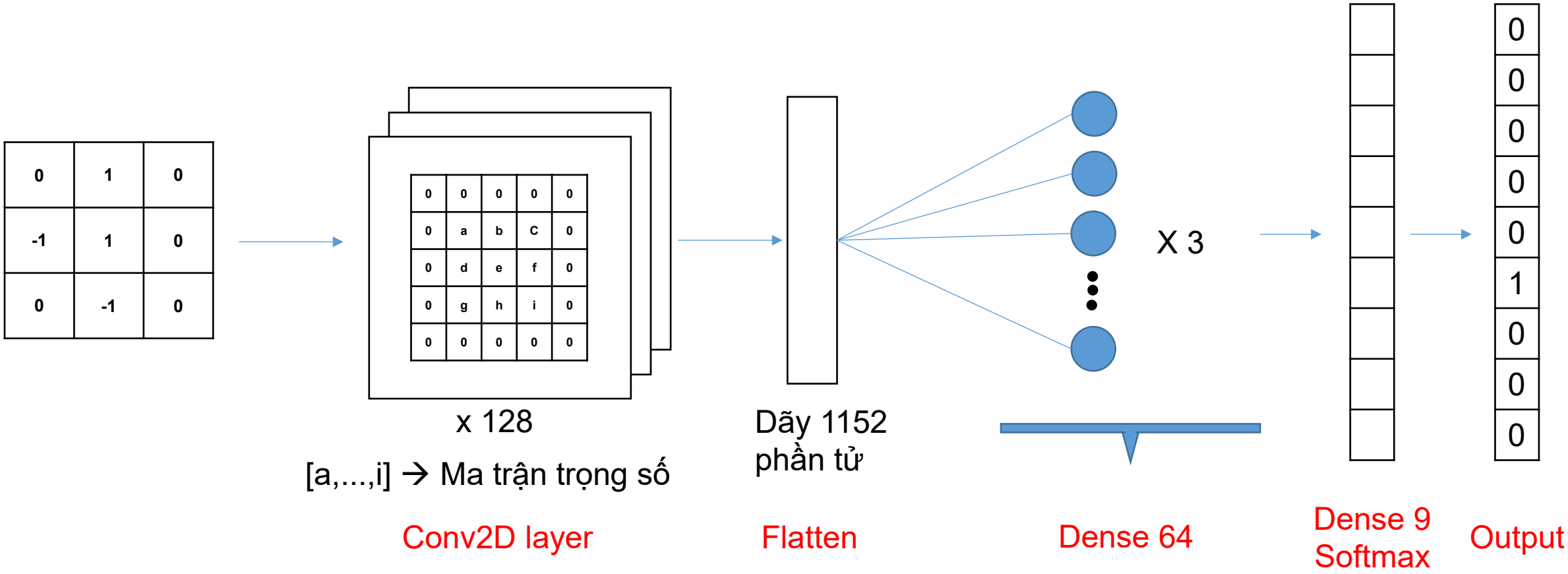
- Kiến trúc của Fast Policy Network

Layer details and parameters (Fast)

Layer	Input shape	Output shape	Parameters
Conv2D (128 filters, 3×3)	(3, 3, 1)	(3, 3, 128)	1,280
Flatten	(3, 3, 128)	(1,152)	0
Dense (64)	(1,152)	(64)	73,792
Dense (64)	(64)	(64)	4,160
Dense (9)	(64)	(9)	585
Total parameters			**79,817**

Deep Learning

- Kiến trúc của Strong Policy Network



Deep Learning

- Kiến trúc của Strong Policy Network

Layer details and parameters (Strong)

Layer	Input shape	Output shape	Parameters
Conv2D (128 filters, 3×3)	(3, 3, 1)	(3, 3, 128)	1,280
Flatten	(3, 3, 128)	(1,152)	0
Dense (64)	(1,152)	(64)	73,792
Dense (64)	(64)	(64)	4,160
Dense (64)	(64)	(64)	4,160
Dense (9)	(64)	(9)	585
Total parameters			**83,977**

Deep Learning

- Vòng lặp huấn luyện và hàm loss.
  - Mỗi epoch là mỗi lần chạy hết dữ liệu mẫu.
  - Hàm loss:

$$L_i = -\sum_{j=1}^{9} y_{ij} \log(\widehat{y_{ij}})$$

4. Key Hyperparameters

Hyperparameter	Value
Input shape	(3, 3, 1)
Conv filters	128
Kernel size	3 × 3
Padding	same
Conv activation	ReLU
Dense hidden layers (Fast)	64 → 64
Dense hidden layers (Strong)	64 → 64 → 64
Output layer	9 (Softmax)
Loss function	Categorical crossentropy
Optimizer	Adam
Epochs	100

5. Comparison

Metric	Fast	Strong
Total parameters	79,817	83,977
Additional parameters	–	4,160
Relative size	1.00×	1.052×
Training time (approx.)	~ 5 min	~ 6 min
Final accuracy (chapter 10)	64.61%	64.73%

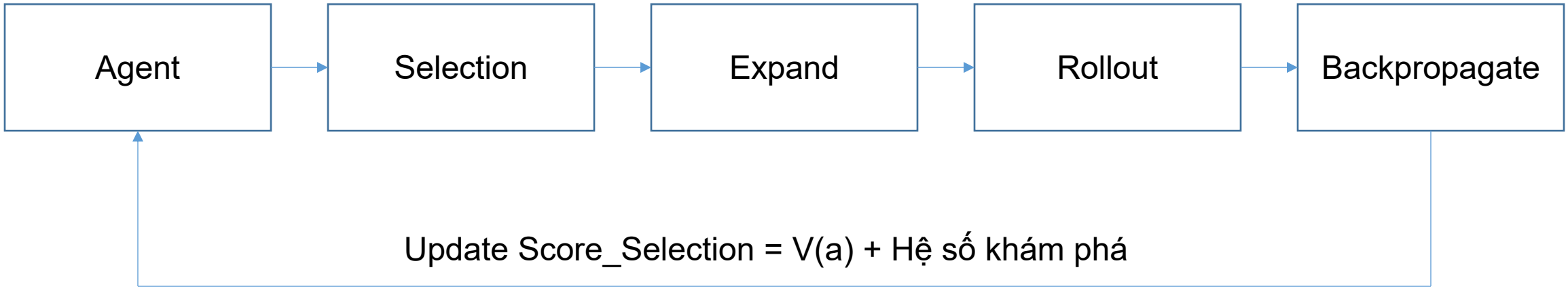
Strong is only one layer deeper than Fast, adding 4,160 parameters (~5.2% more).

Deep Learning

- Vòng lặp huấn luyện và hàm loss.
  - Mỗi epoch là mỗi lần chạy hết dữ liệu mẫu.

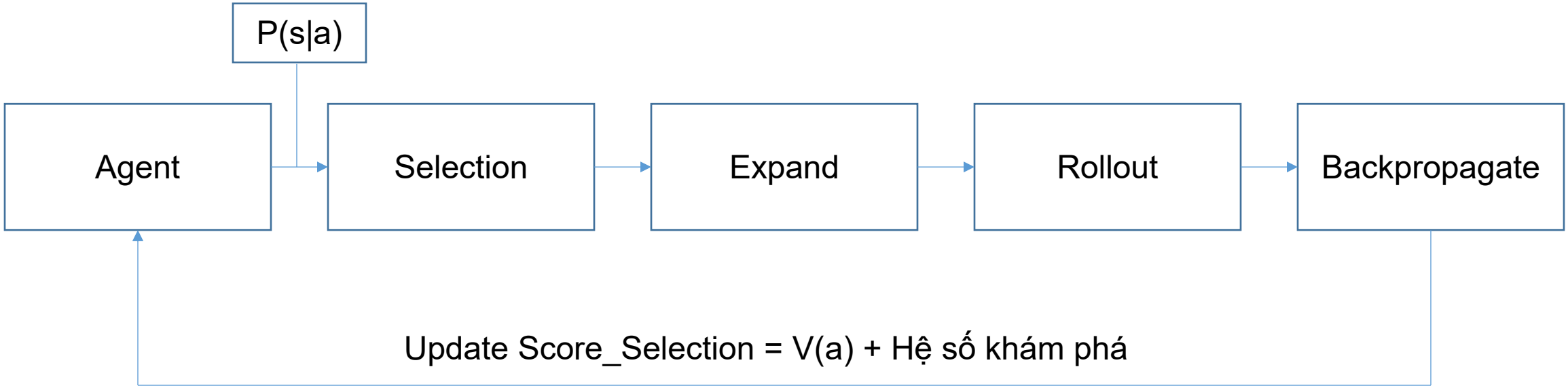
Deep Learning

- Kết hợp giữa UCT và Strong_Policy_Network.
  - Trong quá trình huấn luyện.



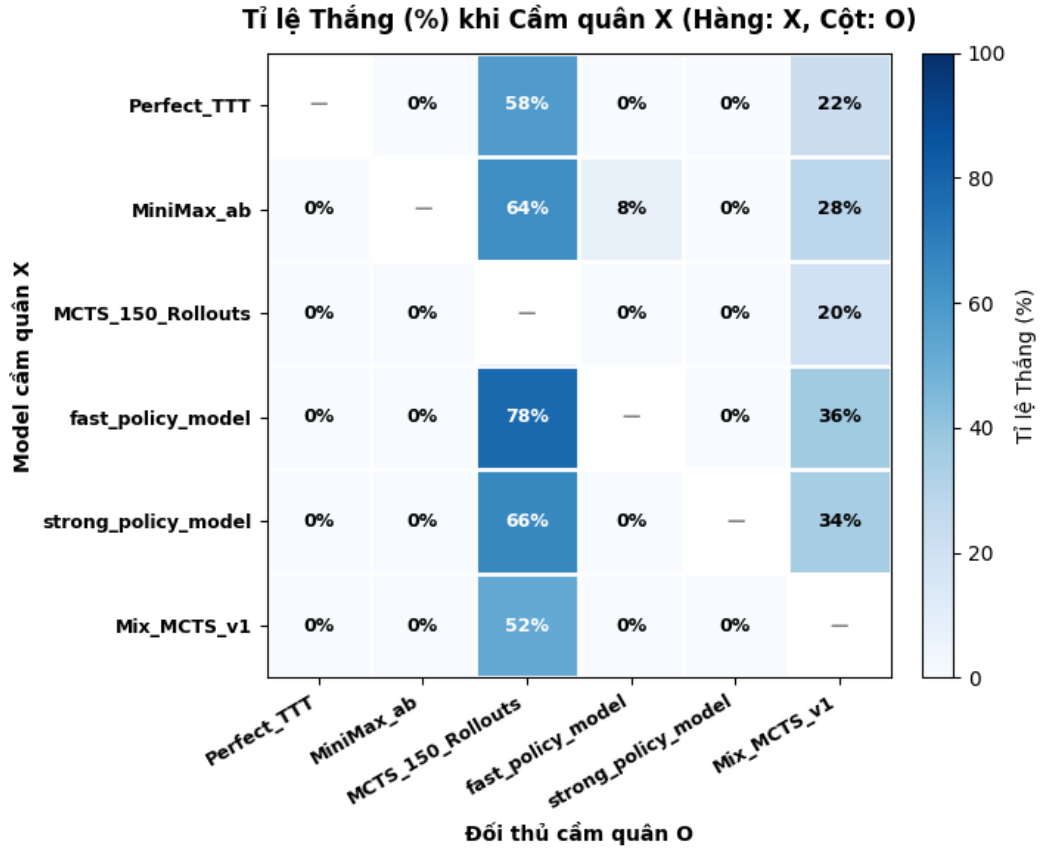
Deep Learning

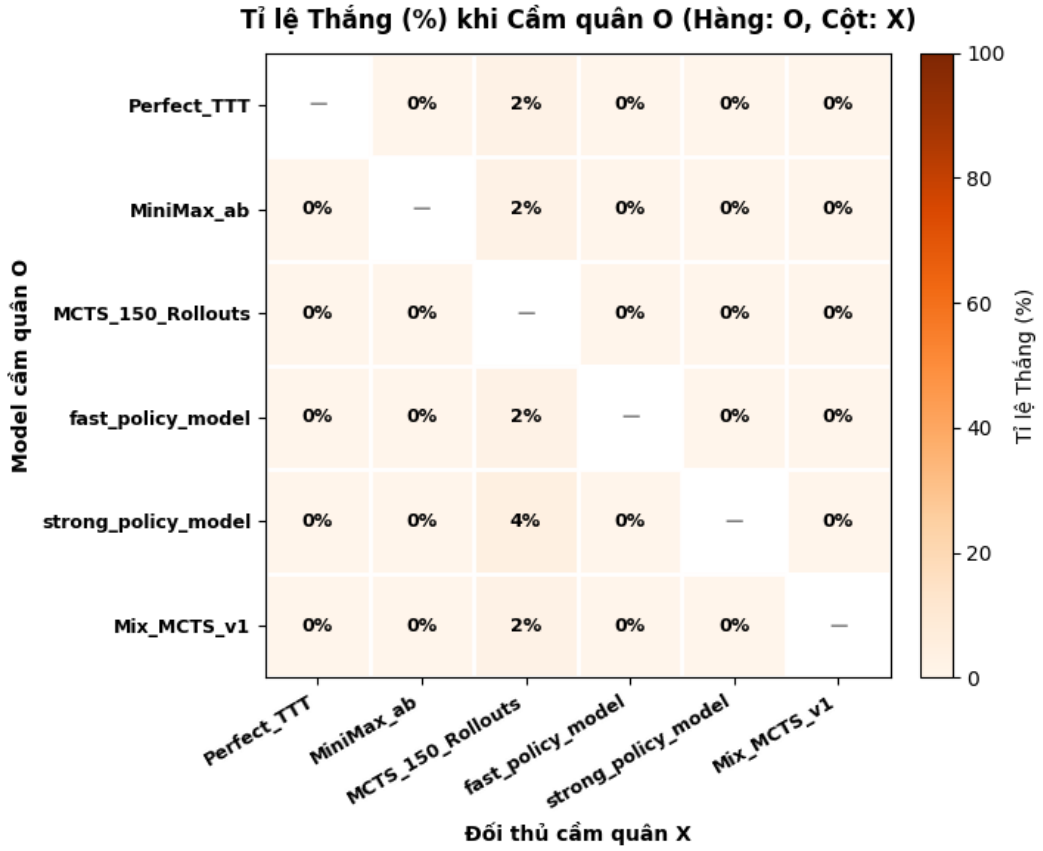
- Kết hợp giữa UCT và Strong_Policy_Network.
  - Trong quá trình huấn luyện.



Deep Learning

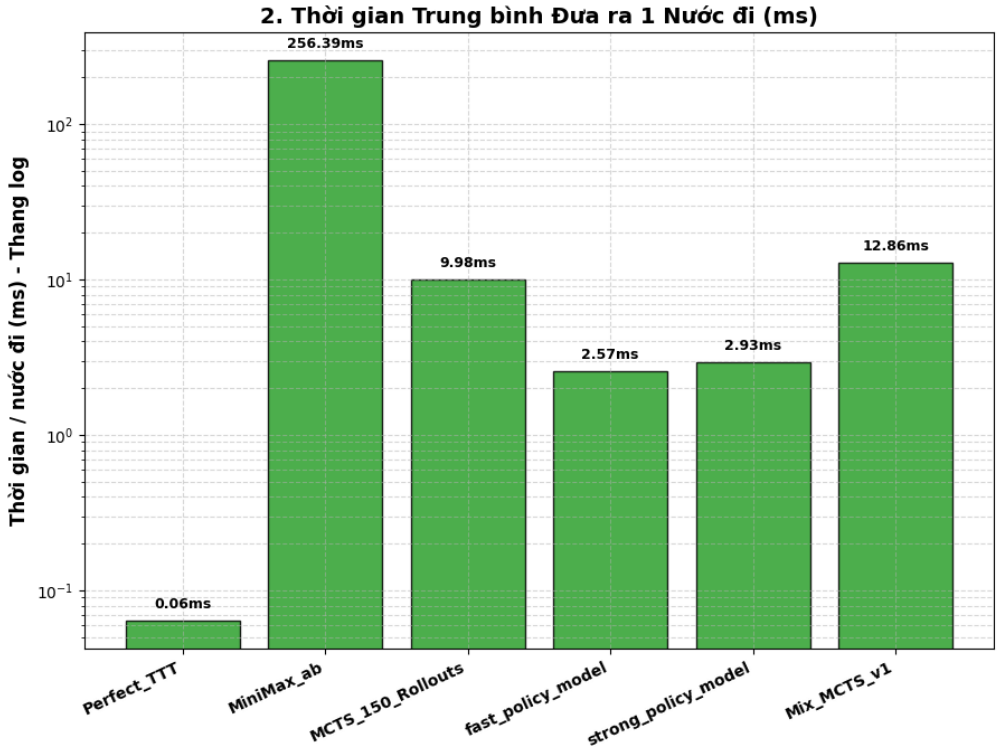
- Kiểm nghiệm kết quả của các mô hình học sâu:

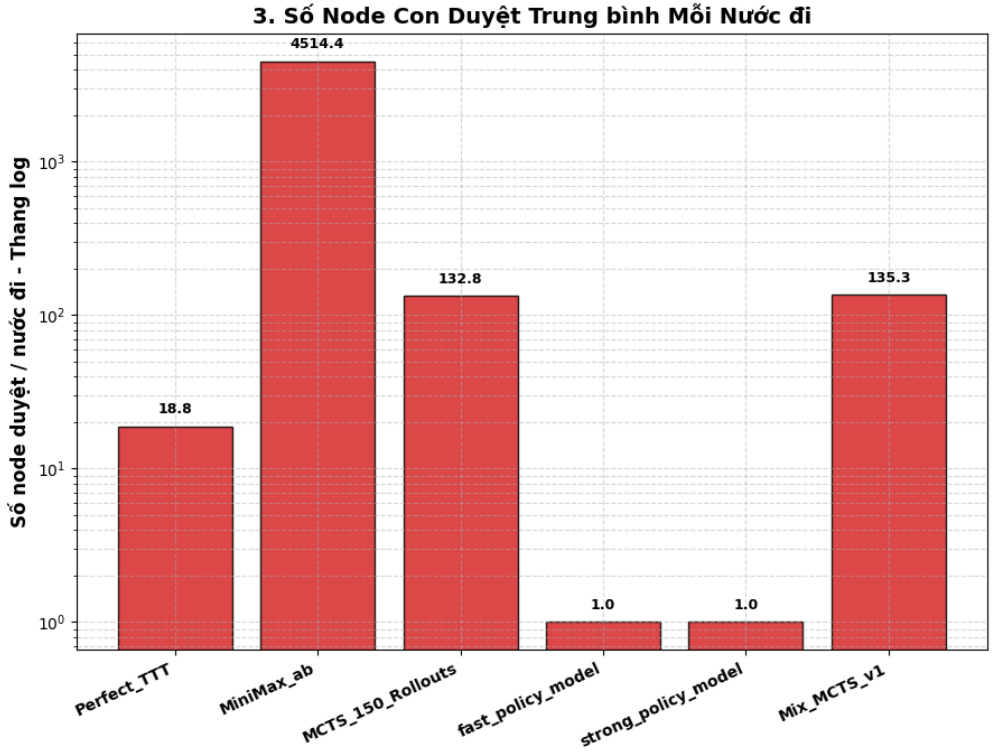




Deep Learning

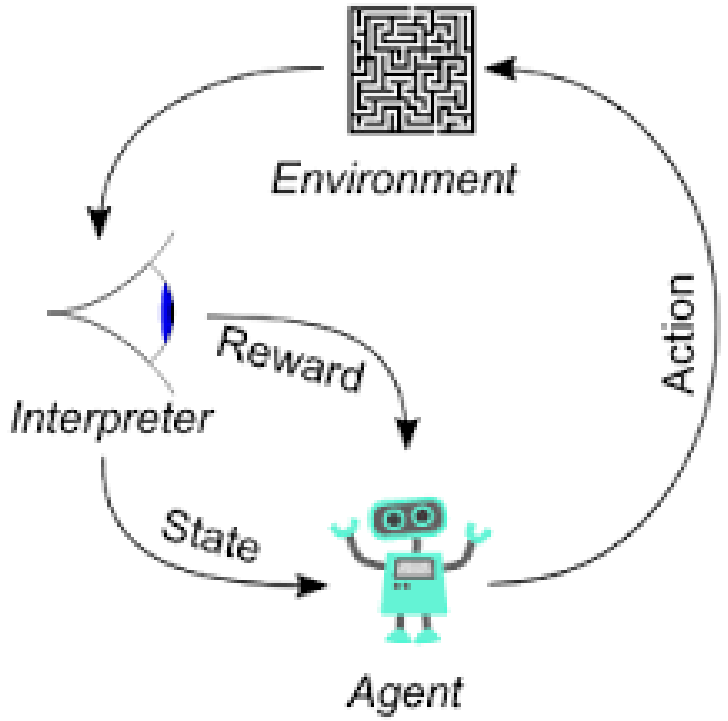
- Kiểm nghiệm kết quả của các mô hình học sâu:





Reinforcement Learning

- Ý tưởng: Đưa Agent là một tác nhân không biết gì hết về môi trường trò chơi, phải tự học để tự nâng cao tri thức bản thân.

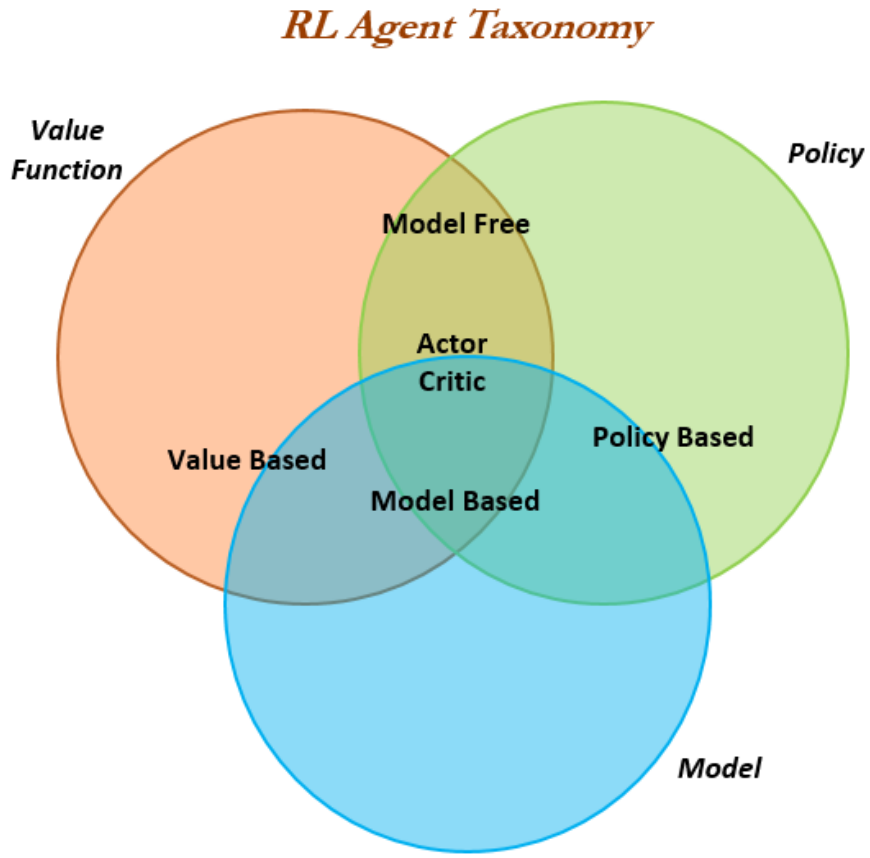


Reinforcement Learning

- Các hướng phương pháp chính:

RL

Học từ cái gì?

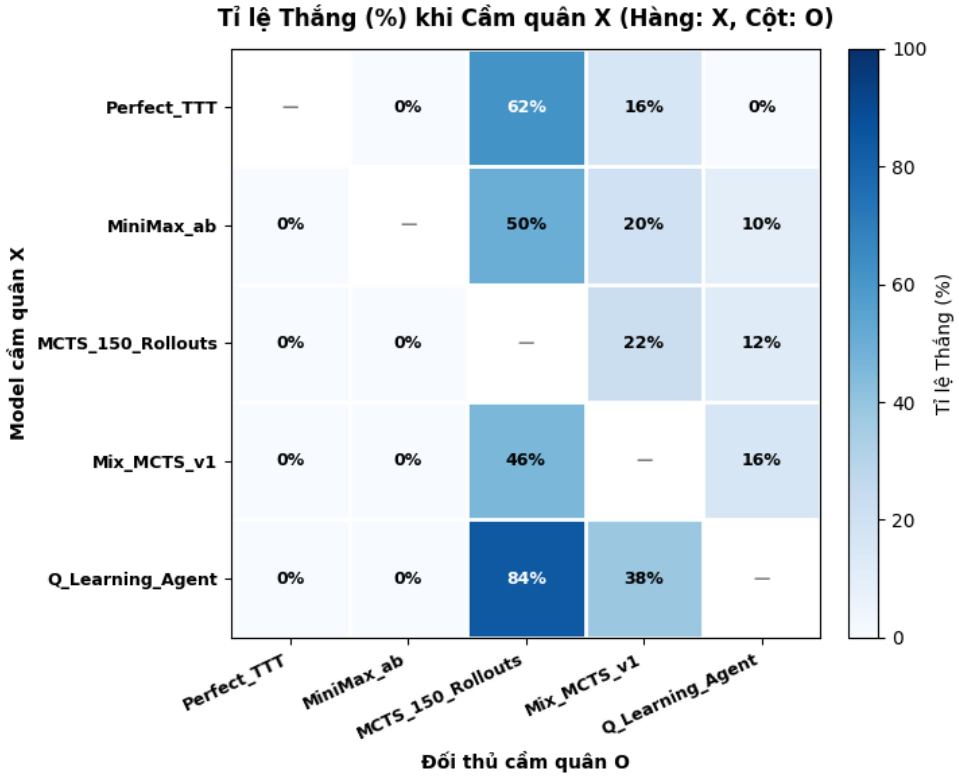


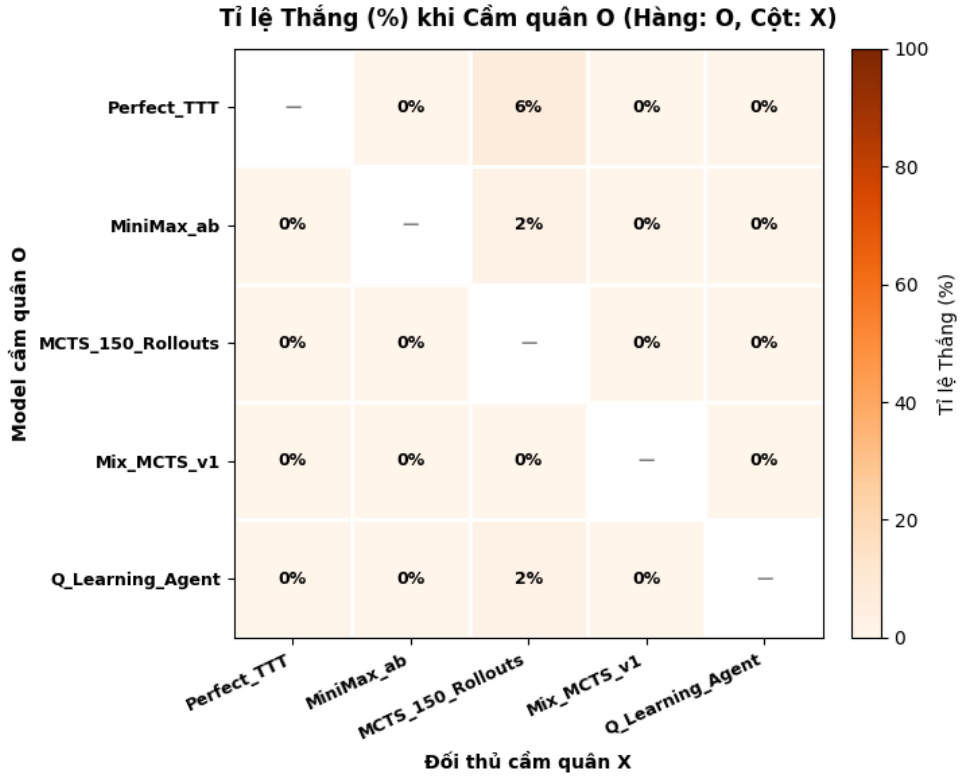
Reinforcement Learning

Node_ID	Depth (Tầng cây)	Best_Action (Ô tối ưu)	Max_Q_Value	Valid_Moves_Count	Valid_Moves_List	Q_Cell_1	Q_Cell_2	Q_Cell_3	Q_Cell_4	Q_Cell_5	Q_Cell_6	Q_Cell_7	Q_Cell_8	Q_Cell_9	Canonical_State
Node_00001	0	9	1.151	9	[1, 2, 3, 4, 5, 6, 7, 8, 9]	0.1351	0.1296	0.1097	0.1817	0.1102	0.06	0.141	0.2965	1.151	(0, 0, 0, 0, 0, 0, 0, 0, 0)
Node_00002	1	5	0.4642	8	[1, 2, 3, 4, 5, 7, 8, 9]	-0.6192	-0.5141	-0.0989	-0.1091	0.4642		-0.738	-0.8093	-0.0937	(0, 0, 0, 0, 0, -1, 0, 0, 0)
Node_00003	1	5	0.3941	8	[1, 2, 3, 5, 6, 7, 8, 9]	-0.1322	-0.5234	-0.6145		0.3941	-0.5124	-0.259	-0.6203	-0.547	(0, 0, 0, -1, 0, 0, 0, 0, 0)
Node_00004	1	5	0.3703	8	[1, 2, 4, 5, 6, 7, 8, 9]	-0.4054	-0.391		-0.7036	0.3703	-0.7289	-0.4121	-0.5067	-0.674	(0, 0, -1, 0, 0, 0, 0, 0, 0)
Node_00005	1	5	0.3176	8	[1, 2, 3, 4, 5, 6, 8, 9]	-0.3187	-0.6568	-0.6029	-0.8198	0.3176	-0.3601		-0.6038	-0.868	(0, 0, 0, 0, 0, 0, -1, 0, 0)
Node_00006	1	5	0.3129	8	[2, 3, 4, 5, 6, 7, 8, 9]		-0.5967	-0.7274	-0.8584	0.3129	-0.8327	-0.5214	-0.4344	-0.5269	(-1, 0, 0, 0, 0, 0, 0, 0, 0)
Node_00007	1	5	0.242	8	[1, 3, 4, 5, 6, 7, 8, 9]	-0.2269		-0.2663	-0.66	0.242	-0.5043	-0.3115	-0.5287	-0.4627	(0, -1, 0, 0, 0, 0, 0, 0, 0)
Node_00008	1	9	0.2355	8	[1, 2, 3, 4, 6, 7, 8, 9]	-0.3134	-0.7746	-0.3451	-0.7842		-0.8791	-0.3532	-0.8954	0.2355	(0, 0, 0, 0, -1, 0, 0, 0, 0)
Node_00009	1	5	0.1014	8	[1, 2, 3, 4, 5, 6, 7, 9]	-0.5873	-0.3037	-0.5028	-0.8781	0.1014	-0.1302	-0.1047		-0.1663	(0, 0, 0, 0, 0, 0, 0, -1, 0)
Node_00010	1	1	-0.2975	8	[1, 2, 3, 4, 5, 6, 7, 8]	-0.2975	-0.6245	-0.7965	-0.7555	-0.3003	-0.8444	-0.3406	-0.6731		(0, 0, 0, 0, 0, 0, 0, 0, -1)

Reinforcement Learning

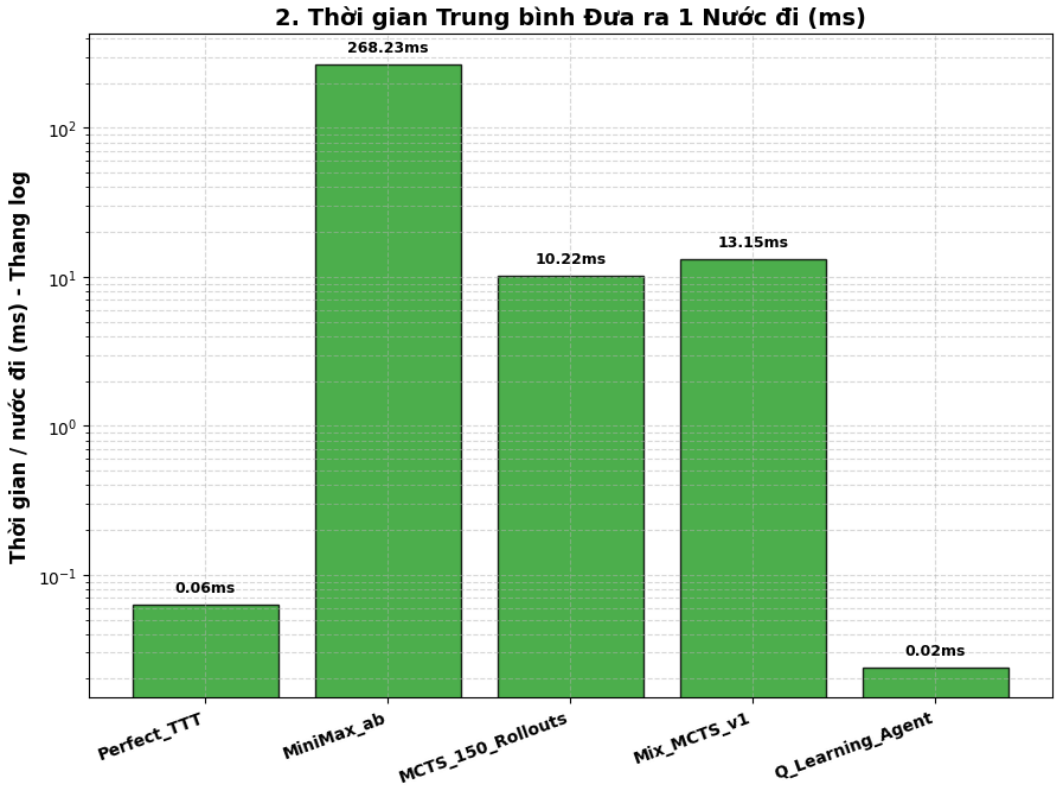
- Kiểm nghiệm kết quả của các mô hình học tăng cường:

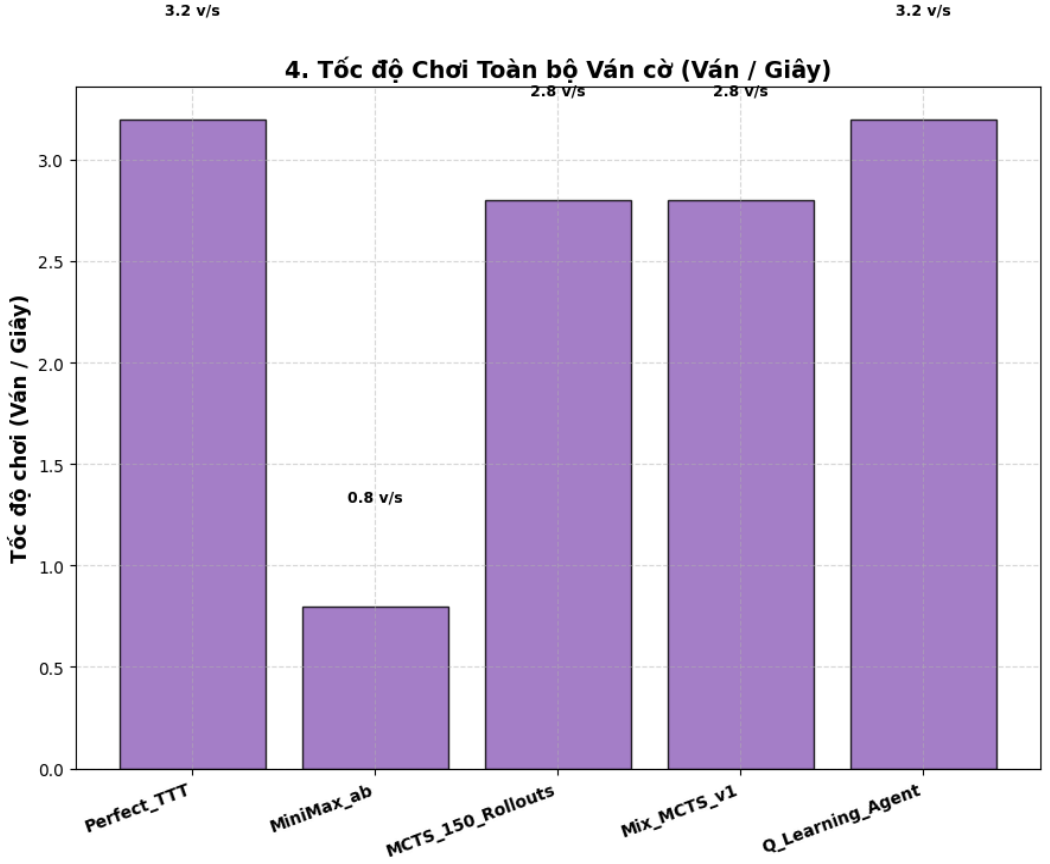




Reinforcement Learning

- Kiểm nghiệm kết quả của các mô hình học tăng cường:





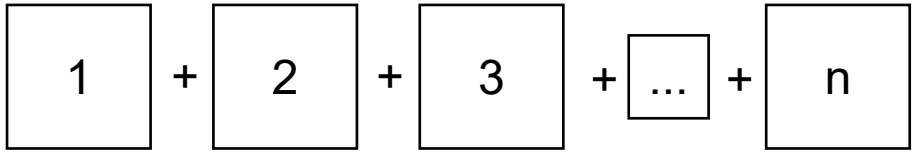
Deep Reinforcement Learning

- Ý kết hợp các đặc điểm và cải thiện khả năng học của mô hình.
  - Đối với Deep Learning: Khả năng ghi nhớ đặc trưng và tính toán thông qua kiến trúc mạng dài hạn.
  - Đối với Reinforcement Learning: Cách thiết kế môi trường tự chơi và học hiệu quả cho huấn luyện để mô hình đạt ghi nhớ và tri thức hiệu quả hơn.

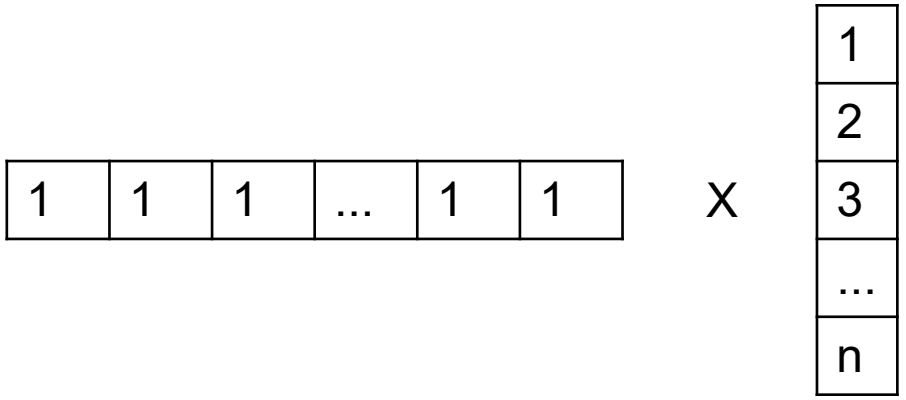
Deep Reinforcement Learning

- Kĩ thuật tính toán đồng thời

Tính toán vòng lặp



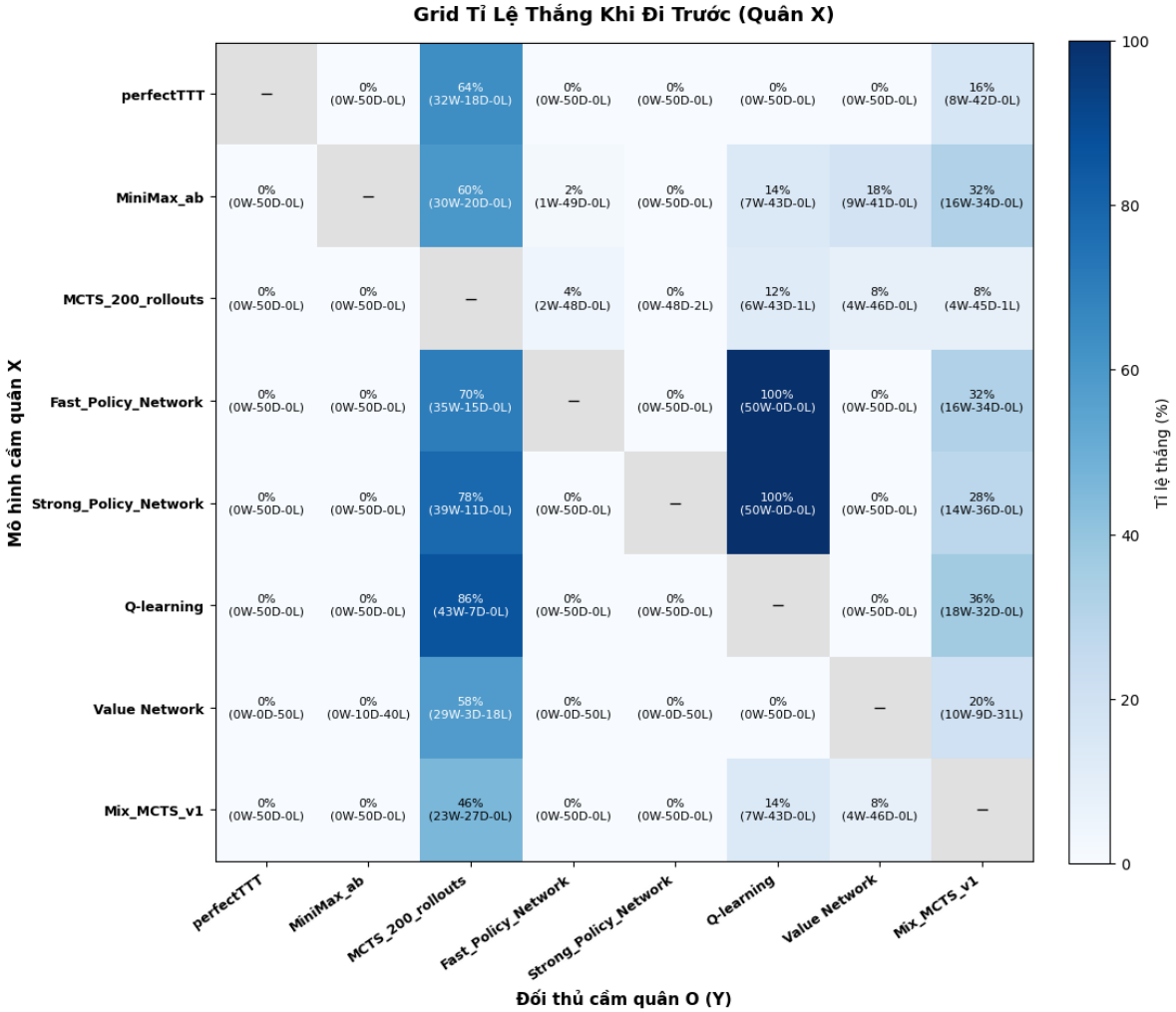
Vectorization

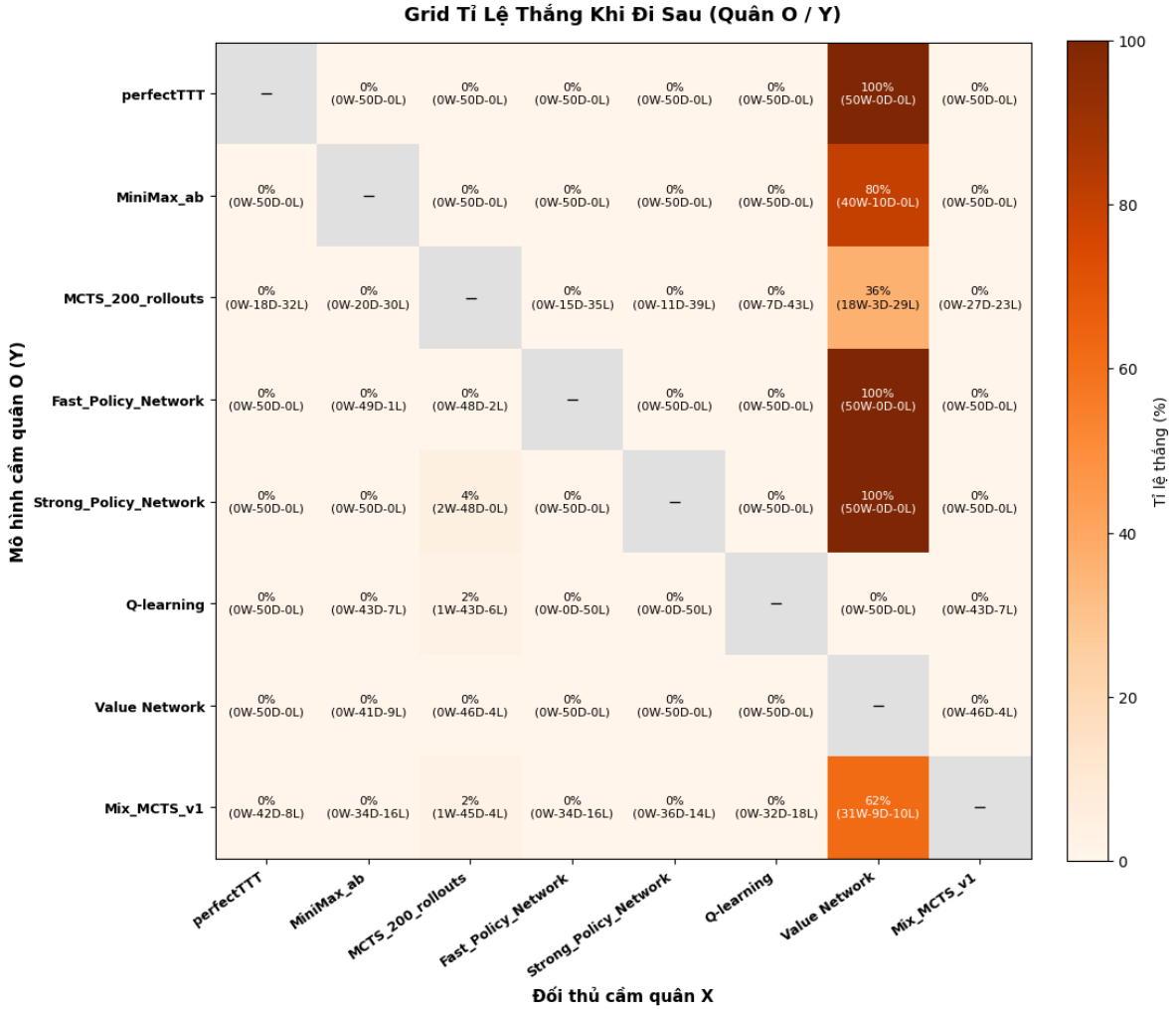


$$a_1b_1 + a_2b_2 + \cdots + a_nb_n$$

Deep Reinforcement Learning

- Kiểm tra hiệu quả giữa các mô hình





Deep Reinforcement Learning

- Kiểm tra hiệu quả giữa các mô hình

